Journal of Information Security and Applications 101 (2026) 104517

Contents lists available at ScienceDirect

Journal of Information Security and Applications

journal homepage: www.elsevier.com/locate/jisa

Model X-Ray: Detection of hidden malware in AI model weights using few shot learning

Daniel Gilkarov [a,b,*], Ran Dubin [a,b]

[a] Department of Computer and Software Engineering, Ariel University, Kiryat HaMada 3, Ariel, 40700, Israel
[b] Ariel Cyber Innovation Center, Ariel University, Kiryat HaMada 3, Ariel, 40700, Israel

ARTICLE INFO

Keywords:
Information hiding
Steganography
Neural networks
Machine learning security
Information security
Digital forensics

ABSTRACT

AI model repositories such as Hugging Face and TensorFlow Hub have become an attractive surface for steganographic malware: attackers exploit the redundancy in float32 weights to embed payloads while preserving model accuracy. Existing AI-model steganalysis methods require tens of thousands of labeled training samples and only detect attacks at high embedding rates ($\geq 50\%$), limiting their practical utility. We address both gaps with a few-shot learning approach. We propose a novel parameter-position-stable image representation, Grayscale-Fourpart (GF), that maps float32 weights to a square grayscale image, and pair it with a metric-learning few-shot CNN detector. The detector trains from as few as 6 model files and consistently flags attacks down to 25% embedding rate, with 6% in some cases. We benchmark against a seven-baseline matrix spanning two prior academic method, the canonical raw-byte 1D-CNN paradigm, and four threshold-based statistics, and identify the conjoint conditions under which the simpler baselines collapse and ours retains accuracy. The trained detectors transfer to novel out-of-distribution spread-spectrum attacks despite training only on LSB perturbations. A deployment-feasibility study shows that GF feature extraction scales linearly to 10^8 parameters at ≈ 0.49 s and ~ 1.77 GiB peak memory, $\sim 352\times$ faster than the strongest prior baseline at the same scale, making this, to our knowledge, the first AI-model steganalysis pipeline practical for repository-scale deployment. The full code framework, including baseline reproductions, is released as open-source.

1. Introduction

Deep Learning (DL) techniques are commonly used for various tasks such as Computer Vision (CV) [1,2], Natural Language Processing (NLP) [3], Robotics [4,5], and more. One of the most significant advancements in Artificial Intelligence (AI) has been the development of Large Language Models (LLMs). LLMs enabled human-like conversations [6], making it easier for people to interact with AI models. This has accelerated the integration of AI models into everyday life, especially among laypeople. Today, enterprises leverage a diverse set of pre-trained models obtained from model repositories, such as those hosted by Hugging Face [7] and TensorFlow Hub [8]. These models serve as powerful starting points, allowing organizations to fine-tune them to meet their specific requirements [9] using significantly less data than required for training a model from zero. But, despite the great benefits of model sharing, they also expose end-users to new and untreated cyber threats that can potentially utilize the DL models for malicious purposes using steganography [10–12], serialization attacks [13], or both [14].

Cyber attackers look for ways to deliver malware to target systems and avoid detection. Among many techniques, one common technique they utilize is steganography [15,16]. Steganography is the art of data hiding, this is a well-researched and developed group of techniques [17] that have been primarily used and researched inside images [18], audio [19], video [20], and text [21]. Among many different steganography techniques, Least Significant Bit (LSB) steganography is a prominent variant of steganography [22]. These techniques embed data inside the least significant portions of the cover data, and in doing so, the noticeable effect is minimized. Similar to many well-known LSB steganography attacks on various digital media types, LSB steganography can be used in DL models. It can be exploited for malicious purposes e.g. for delivering malicious payloads [10,11,23]. However, in contrast, DL models are often much larger, and therefore, they may offer much

* Corresponding author.
E-mail addresses: daniel.gilkarov1@msmail.ariel.ac.il (D. Gilkarov), rand@ariel.ac.il (R. Dubin).

https://doi.org/10.1016/j.jisa.2026.104517

Available online 30 May 2026
2214-2126/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

greater hiding capacity than image, audio, or text data. The increased data transmission capacity may enable an attacker to use attacks that weren't possible before or that are harder to carry out. These techniques take advantage of redundancy, and DL models often use float parameters (32/16 bits), which offer great precision but may sometimes be redundant (see Appendix B). To overcome these attacks, several works [24,25] suggested ways to disrupt the attacks by modifying the parameters or their representations by using quantization or random bit modification [24] or by fine-tuning [25]. These methods can possibly be countered by more sophisticated data-hiding techniques that utilize self-correcting codes to resist data corruption. In [14], novel AI model steganography techniques utilize low-density parity-check codes, demonstrating resilience to parameter pruning and fine-tuning. Therefore, a detection approach is needed.

Steganalysis techniques are a set of methods aimed at detecting hidden messages in cover data, and they are designed to counter steganography. These detection techniques are an alternative strategy to zero-trust methods that aim to damage the payload by changing the model; instead, the aim is to detect the presence of hidden data. The detection approach has certain benefits over the techniques mentioned earlier: by using detection to filter potentially unwanted files, model repositories or end-users can avoid tampering with the models, which can cause unwanted side effects. In addition, detection may prove effective in cases where the data-hiding methods have some resilience to deliberate data corruption [14], as mentioned above. This article explores and develops steganalysis techniques for AI model LSB steganography attacks that can be applied in real-world scenarios.

One major drawback in traditional ML and DL learning algorithms is that they often require large amounts of training data to avoid overfitting [26]. This can be problematic for real-life scenarios where we want to detect steganography attacks inside AI models since they vary in size, architecture, task, and many more factors. The major obstacle we encounter is data scarcity. We require attacked models to research AI model LSB steganography, but no public sources exist. To this end, we build upon the methodology and framework for simulating attacks and creating datasets for research published in previous work [23,27]. In addition, we focus on DL techniques that are specialized for use in data-scarce scenarios.

One-shot Learning (OSL) and Few-Shot Learning (FSL) are techniques developed to use DL in areas with significantly less training data than traditional DL models require. The core concept is that a model is trained only using one or a few samples per label. OSL and FSL are commonly utilized for CV tasks [28–30]. There are different approaches to tackling OSL and FSL - augmenting training data to get more samples [31], fine-tuning pre-trained models [32], training embedding models to project embedding samples to a smaller embedding space [33], etc. A popular FSL approach is called metric-learning [34]. In metric learning, samples are clustered based on their distance from other samples. In a classification context, a sample is labeled based on proximity to other labeled samples according to a certain distance function (like l_1 or l_2). The widely-known KNN algorithm is a canonical example. Metric learning is often paired with an embedding mechanism that embeds samples to a lower dimension (for example, a convolutional network [35]). This is called embedding learning. In the training phase, the goal is to minimize the distance between similar samples and maximize the distance between dissimilar samples (in classification, same-label samples are "similar" and vice versa for samples with different labels).

FSL techniques have been found to have extensive applications in the field of computer vision. Convolutional Neural Networks (CNN) are one of the most popular neural network types used in this domain, and a significant amount of research on FSL involves embedding learning using CNN networks as embedding functions with image data. Some researchers have also transformed data into image representations and applied OSL and FSL techniques to them [36]. In this article, we apply FSL techniques to AI models by creating image representations from them, allowing us to apply the rich and deeply developed image FSL techniques to our field. We use metric learning by training a CNN to use as an embedding function and we show that we can effectively classify unseen samples using 2 different evaluation techniques. We compare results and identify different use cases for the different methods.

1.1. Our contribution

The key contributions of our research are enumerated below:

- **Grayscale-Fourpart (GF) image representation.** We propose a novel image representation of float32 model weights that splits each parameter into four 8-bit segments and tiles them into a 2×2 grid, producing a parameter-position-stable square grayscale image (Section 3.3). The representation is designed to enable reuse of CNN-based image-domain methods on AI-model weight tensors, and to our knowledge is the first weight-domain image representation specifically designed for steganalysis of AI models.
- **Few-shot metric-learning steganalysis pipeline.** We adapt OSL CNN [35] and SRNet [37] as feature extractors over the GF representation, train them with triplet loss and ℓ_2-normalized embeddings, and classify with centroid and 1-NN procedures (Section 3.4). The pipeline is the first AI-model steganalysis method that operates in a few-shot regime, training from as few as 6 model files vs. 100 [38] or ∼40000 [23] required by prior work.
- **Weighted detection metric (WM).** We define a weighted evaluation metric (Section 4.6) that prioritizes correct classification of low-embedding-rate attacks - the regime an attacker would realistically use - aligning quantitative evaluation with the practical detection objective rather than reporting unweighted classification accuracy across the full attack range.
- **Comprehensive empirical comparison.** We benchmark our method against a seven-baseline matrix (Section 4.1): two prior academic methods (B1 Gilkarov et al., B2 Yin et al.), one raw-byte 1D-CNN baseline (B3 MalConv-lite [39]) included to compare the image route against the raw-byte sequence paradigm, and four naive threshold-based baselines (B4-B7) representing classical malware-analysis and steganalysis statistics. The matrix is evaluated under matched data-budget and architectural-homogeneity protocols, isolating exactly when each baseline succeeds and when it fails.
- **Cross-architecture, cross-attack generalization study.** We show that the trained detectors generalize to held-out CNN architectures across the DenseNet, ResNet, MobileNet, NASNet, Xception, EfficientNetV2, and Inception families up to ∼89M parameters, and to an out-of-distribution spread-spectrum attack (MaleficNet [14]) despite training only on X-LSB-Fill perturbations (Section 4.8). The OSL-CNN-vs-SRNet ID/OOD ranking inversion observed on MaleficNet is grounded in a mechanism analysis based on the two architectures' source-paper design choices (Section 4.8, Experiment 2.5).
- **Deployment-feasibility analysis.** We measure the cost of GF feature extraction and FSL inference up to 10^8 parameters, demonstrate $O(n)$ linear scaling at ≈ 0.49 s and ~ 1.77 GiB peak memory at $n = 10^8$, and contrast this with the ∼352× slower B2 baseline at the same scale (Section 7). To our knowledge this is the first end-to-end deployment-cost study for AI-model steganalysis.
- **Open-source reproducibility artifact.** We release the full pipeline (data creation, X-LSB-Fill and MaleficNet attack replication, image representation, FSL training and evaluation, baseline reproductions B1-B7) as an open-source software framework [40].

The remainder of this paper is structured as follows: Section 2 provides background knowledge. Section 3 describes the proposed methodology for data creation, pre-processing, and classification. Section 4 presents the experimental results and analysis. Section 5 discusses limitations and future work. Section 6 reviews the latest works related to this paper. Section 7 analyzes the practical deployment of our method. Finally, conclusions are drawn in Section 8.

2. Background

2.1. X-LSB-Attack

X-LSB-Attack [23] embeds a malware sample m inside an ordered collection of float numbers. The attack works by using LSB substitution similar to other basic LSB steganography methods in other types of data (images [41], audio [42], etc.) The attack embeds m in the X least significant bits (mantissa/fraction part [43]) of the float numbers, similarly to LSB-substitution procedures introduced in previous works [10,11]. We use an extra version of X-LSB-Attack called X-LSB-Attack-Fill [23] that fills the whole cover data by repeating m or using the largest prefix of it that fits in the X least significant bits of all parameters in the model. As in prior work [23], we operate on float32 weights, whose mantissa size is $ms = 23$ bits. Since changes outside the mantissa are easily noticed [23] (see Appendix B), we focus on embedding data in the mantissa regions. See Algorithm 2 for pseudo-code of **X-LSB-Attack** and Algorithm 3 for pseudo-code of **X-LSB-Attack-Fill**.

2.1.1. Rationale for using X-LSB-Attack-Fill

In our experiments, we exclusively use the X-LSB-Attack-Fill variant rather than the basic X-LSB-Attack for several methodological reasons. The basic X-LSB-Attack introduces unmanageable variability when the payload size is smaller than the available embedding capacity: attackers can choose arbitrary embedding locations within the model, position the payload at the beginning, middle, or end of the weight sequence. This positional variability would exponentially increase the number of attack variants we must consider, making systematic evaluation infeasible.

X-LSB-Attack-Fill eliminates this variability by completely utilizing the available embedding capacity. In our experiments, we select malware payloads whose size is greater than or equal to the model's embedding capacity at a given LSB depth X. This ensures that the entire mantissa region is modified in a deterministic manner, removing positional ambiguity while still representing realistic attack scenarios. In practice, attackers seeking to maximize payload delivery or maintain consistency across model updates would similarly utilize the full embedding capacity. Furthermore, this approach provides a consistent baseline for evaluating detection methods across different model architectures and attack severities, as the embedding pattern depends only on X and the model size, not on arbitrary placement choices.

This design choice aligns with our goal of training robust, generalizable detection models. By focusing on the most consistent attack variant, we avoid overfitting to specific payload positioning strategies and instead train our models to detect the fundamental signatures of LSB manipulation in the mantissa region.

2.2. Embedding rate

The embedding rate is a measure for steganography attacks. It measures how much (%) of the cover data is embedded with hidden data. In the context of steganography inside AI model weights, this can be expressed by

$$ER = \frac{n_b}{n_w \cdot s}$$

where n_b denotes the number of bits in the embedded payload, n_w denotes the number of weights in the cover model, and s denotes the size of the models' weights (e.g., 32 if the parameters are float32). Since our experiments use X-LSB-Attack-Fill (Section 2.1) we can express it directly:

$$ER = \frac{n_w \cdot X}{n_w \cdot s} = \frac{X}{s}$$

3. Methodology

This section illustrates the methodologies for developing FSL models to detect LSB steganography attacks in AI models. The main workflow in this article consists of three phases: dataset creation (Section 3.1), model training (Section 3.4), and model evaluation (Section 3.4).

3.1. Dataset creation

Datasets for training FSL models are created by following 3 steps: model collection, embedding malware, and pre-processing. Every model collection is processed into a binary classification dataset of benign (labeled 0) and malicious models (labeled 1). See Fig. 1 for an illustration.

Model collection. Model Zoos (MZ) are a population of models of the same architecture trained on the same task and can have only one model. We define a Model Collection (MC) as a collection of MZs. MCs for FSL are created by downloading pre-trained popular models from online model hubs like Tensorflow-Hub [8] and grouping them based on their size.

Embedding malware. Using the MC of pre-trained benign models, we create attacked versions of it by attacking every model in the collection using the same attack. We use the X-LSB-Attack-Fill method (Section 2.1) to attack the models using a malware sample m from MalwareBazaar [44]. We embed multiple malware payloads or randomly generated binary payloads for large models (>300M parameters). The float mantissa region has ms bits, so we use X-LSB-Attack-Fill with $1 \leq X \leq ms$ - this results in ms different attacked model collections of different attack severity, a lesser X value means a more subtle, harder-to-detect attack.

Pre-processing. We apply pre-processing to our data to make it more suitable for our FSL models (siamese CNNs that work with constant-sized images). We developed techniques for creating images from float numbers (model parameters, etc.). See Section 3.3 for an in-depth explanation of the procedure. After we convert our data into image form (8-bit grayscale), we reshape (downscale or upscale) the images into a constant predetermined size and normalize the 0–255 integer values to the 0–1 real values range - this is common practice for training CNNs [45].

Dataset hyperparameters. Using our methodology, datasets can be created in a large variety of ways. Every dataset D will have a set of hyperparameters $hp = (MC, (X, m), (I, sh))$ that influenced its creation. Each step in the creation procedure uses the hyperparameters. MC is the Model Collection of MZs from which the dataset was created. (X, m) are used in the embedding procedure - X is the amount of LSBs we chose to override and m is the malware sample that was embedded. (I, sh) are used in the pre-processing phase, I is the image transformation used (See 3.3) and sh is the final shape all images were reshaped into.

3.2. Train/test split

We split the processed image datasets into train and test sets using the following methodology:

$$Let\ MZS = \{MZ_1, MZ_2, \ldots\}$$

The model zoos from which we created our datasets. Each model zoo has models of the same architecture trained on the same data task. We pick some subset of the model zoos and set:

$$MZS_{train} \subseteq MZS$$

$$MZS_{test} := MZS \setminus MZS_{train}$$

We compile the training dataset by taking all models (benign and attacked) of the model zoos in MZS_{train}, and the test dataset is compiled similarly with MZS_{test}. The split is deliberately performed to help prove the trained model's robustness by testing it against models of architectures that weren't seen during training.

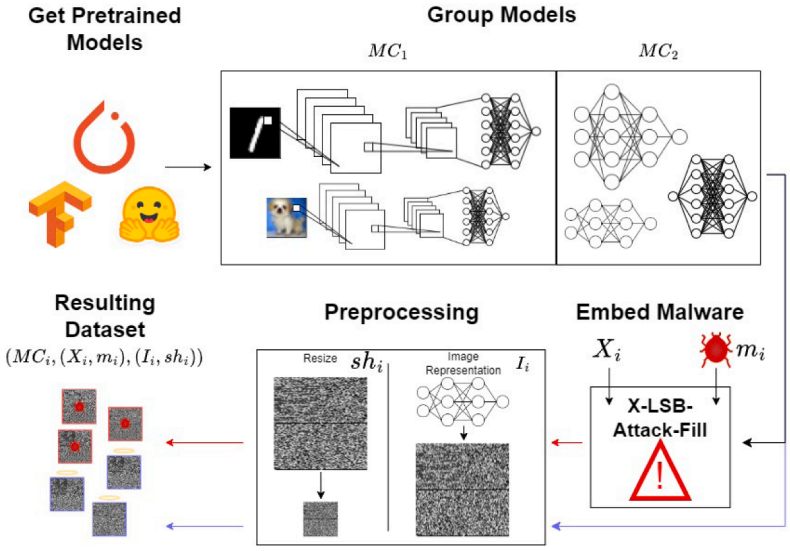


Fig. 1. Overall dataset creation process (Section 3.1). First, we get pre-trained models from model repositories and create groups of model zoos based on model size. We call them Model Collections. In this example, we have MC_1 with 2 CNNs and MC_2 with 3 different DNNs. Second, we use each model collection MC_i to create an attacked version of it by using X-LSB-Attack-Fill (Section 2.1) with some value X_i and malware payload m_i. Finally, the original and attacked MCs (colored blue and red respectively) go through a pre-processing phase. In this example, we calculate some model image representation I_i (Section 3.3) and reshape all resulting images to shape sh_i. The process results in 2 datasets of benign and malicious model image representations created from MC_1 and MC_2. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

3.3. AI model image representation

This section will discuss our methods for creating image representations from float model weights. We introduce a novel image representation called Grayscale-Fourpart.

The Grayscale-Fourpart image representation works on float32 data specifically. It is created by dividing each 32-bit model weight $w := b_{31} \cdots b_0$ to 4 8-bit parts: $p_0 := b_{31} \cdots b_{24}, p_1 := b_{23} \cdots b_{16}$ and so on. Each 8-bit long binary string (byte) is used as an unsigned int8 value with values in the range 0–255 like standard 8-bit images and serves as one pixel. 4 images are created by stacking the model weight parts p_1, p_2, p_3, p_4 from all weights (all p_1's, all p_2's, …) and shaping the stacked vector of byte values into square shapes (adding zero padding if needed). The 4 images are stacked side to side to form the final image. See Algorithm 1 and Fig. 2.

Algorithm 1 Grayscale-Fourpart.

Input: Vector of float32 data in binary form $BINW$ of shape $(n, 32)$

Output: 8-bit Grayscale Image I

1: Divide $BINW$ into 4 parts (each of shape $(n, 8)$):
$BINW_j := BINW[:, 8(j-1) : 8j]$ for $1 \le j \le 4$
2: let $\hat{n} := \lceil \sqrt{n} \rceil$ the nearest square of n (from above)
let PAD = zero vector of shape $(\hat{n}^2 - n, 1)$
3: Calculate unsigned int8 matrices (each of shape $(\hat{n}, \hat{n})$):
$I_j := uint8(BINW_j) + PAD$ reshaped to shape $(\hat{n}, \hat{n})$ for $1 \le j \le 4$
4: **Return** $I := \begin{bmatrix} I_1 I_2 \\ I_3 I_4 \end{bmatrix}$

3.3.1. Design rationale for Grayscale-Fourpart

The Grayscale-Fourpart image representation is designed to bridge AI model weight analysis with the mature field of image processing and classification. Our design choices are motivated by the following principles:

Compatibility with standard image formats. Standard digital images are either grayscale (1 channel, with pixel values in the range 0–255) or RGB (3 channels, with pixel values in the range 0–255 per channel). By segmenting float32 weights into byte-sized components, we ensure compatibility with established image processing pipelines and enable the direct application of well-studied image classification algorithms without requiring specialized preprocessing.

Lossless representation with natural byte alignment. Since float32 data is 4 bytes (32 bits) in length, we disassemble each float32 no. into 4 byte-sized (8-bit) segments. Each segment naturally maps to a single grayscale pixel value (0–255), creating a lossless representation that preserves all information from the original model weights. This 4×8-bit segmentation aligns perfectly with byte boundaries, avoiding the computational overhead and compatibility issues that would arise from alternative segmentations such as 2×16-bit or 1×32-bit, which would require additional conversion steps to interface with standard image processing libraries.

Square image construction for optimal CNN processing. We construct a standard square grayscale image by stacking the four sub-images (one for each byte segment) together in a 2×2 grid configuration. This square format is optimal for convolutional neural networks, which typically expect square or near-square inputs. The resulting image can be processed using standard image operations such as resizing, and normalization, and can be directly fed into any image classification architecture.

Granularity for detecting steganographic patterns. The 4×8-bit segmentation provides fine-grained spatial granularity in the resulting image representation. LSB steganography attacks modify the mantissa region bit-by-bit (see Section 2.1), and our byte-level segmentation allows CNNs to detect subtle bit-level patterns across different regions of the float32 structure. Coarser segmentations (e.g., 2×16-bit or treating the entire float32 as a single unit) would blend these patterns together, reducing the spatial resolution available for the CNN to learn discriminative features.

Preservation of float32 structural information. Float32 numbers have distinct structural regions: 1 sign bit, 8 exponent bits, and 23 mantissa bits (see Appendix A.2). Our 4×8-bit division naturally preserves this structure in the image representation. The first byte (containing the sign bit and 7 exponent bits) is separated from the three subsequent bytes (containing the 23-bit mantissa), where steganographic attacks

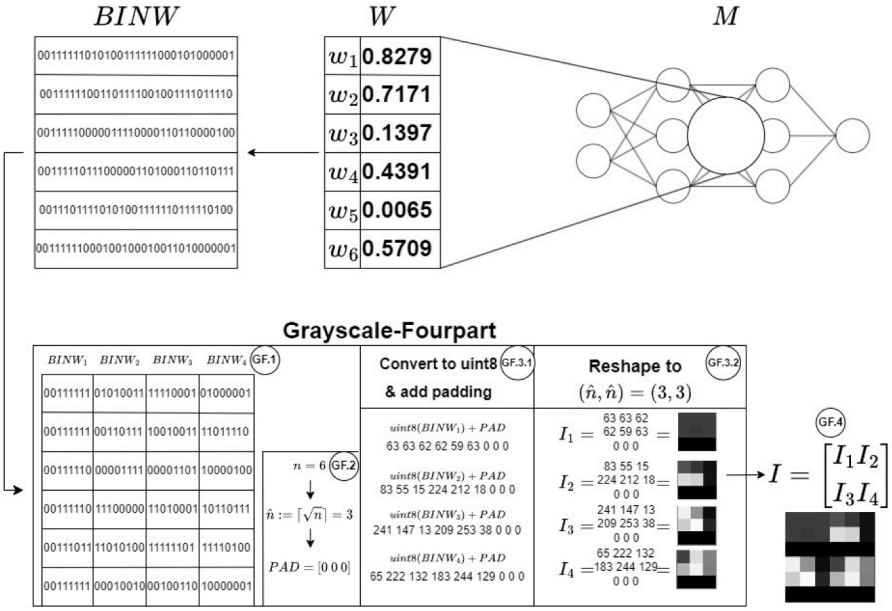


Fig. 2. Illustration of a technique for creating image representations from binary form float32 data ($BINW$). The technique is used to create images from model weights and gradients. The Grayscale-Fourpart (GF) technique splits every float32 into 4 segments (GF.1) - resulting in 4 vectors of 8-bit binary values ($BINW_1, BINW_2, \ldots$). To turn the vectors into a square shape, the closest square root from above ($\hat{n}$) is calculated, and zero padding is applied as needed (GF.2). Each byte vector is converted from binary form to uint8 (0–255) and the zero padding is added (GF.3.1). The vectors are reshaped to a square matrix - each acting as an image (GF.3.2), Finally, the Images are stacked to form the output grayscale 8-bit image I (GF.4).

primarily occur. This structural separation enables CNNs to learn region-specific attack signatures, as modifications to different float32 regions have fundamentally different characteristics and impacts on model behavior (see Appendix B).

Why an image representation rather than a raw byte/text sequence. An alternative to the image route would be to feed the model file as a flat sequence of bytes to a 1D sequence model, in the spirit of MalConv [39] for executable malware. We argue this is a poor fit for steganalysis of model weights for two reasons. First, the position semantics differ: in an executable, byte N may encode code in one file and data in another, so the model must learn to interpret each byte from local context [39]. In an architecture-matched model zoo, by contrast, parameter index N refers to a comparable quantity across models, so the same pixel position in our representation maps to the same weight role across samples; this position stability is what makes a CNN's spatial inductive bias informative for our task. Second, the available supervision differs by orders of magnitude: MalConv-style raw-byte models learn a byte embedding from $4 \times 10^5 - 2 \times 10^6$ labeled binaries [39], whereas our few-shot setting trains on as few as 6 models (Section 4). Mapping weights into an image and reusing image-pretrained CNN backbones (Section 3.4) sidesteps the need to learn a byte-level representation from scratch, which is what makes detection from such small training sets feasible. We discuss the relationship to MalConv and prior image-based malware classifiers further in Section 6.

By combining these design principles, Grayscale-Fourpart enables us to leverage the rich ecosystem of image-based few-shot learning techniques while maintaining a complete, efficient, and interpretable representation of AI model weights.

3.4. Convolutional embedding network

This section outlines our methodology for training a CNN to learn embeddings. We used the triplet method [46] with ℓ_2 distance for the training strategy. We take our train data and construct all possible (anchor, positive, negative) triplets, which in our case is based on the labels (i.e. (benign, benign, malicious) and (malicious, malicious, benign)). The CNN feature extractor's output is ℓ_2-normalized before the triplet distance is computed, so the loss operates on the unit sphere. We train the CNN feature extractor using Adam [47]. We experiment with three different training strategies:

1. Early Stopping (ES): train for one epoch.
2. Standard Training (ST): train for five epochs.
3. Until Below (UB): train for up to 100 epochs, and stop training if train loss enters a fixed loss-window, e.g. $[0.5, 1.25]$.

The concrete numeric hyperparameter choices that instantiate this recipe in our experiments (margin, learning rate, batch sizes, regularization, image preprocessing) are reported in Section 4.3.
Testing phase: Model evaluation is done with 2 different procedures that show competitive results. We focus on methods that can be practically used.

Centroid. We classify a new sample using the centroids of the benign and malicious training data. This is common practice in FSL [48]. Let D_0 and D_1 denote them, let I denote the image to be labeled, and f to be the trained CNN. A centroid of some data D_i is given by $c_i = \frac{1}{|D_i|} \sum_{d \in D_i} f(d)$. I is then given the label $argmin_{i \in \{0,1\}}\, l_2(f(I), c_i)$. See Fig. 3.

KNN. We classify a new sample using a similar process to the centroid technique, but we apply KNN using all embeddings of the benign and malicious training data instead of the average embeddings.

4. Experiments

In this section, the experimental results are laid out and analyzed. The experiments are carried out by exercising the data creation and classification methodology discussed in Section 3. In each experiment, we tried out different MCs that we had compiled. We train models using some dataset D. We evaluate them on all attack severities ($1 \leq X \leq s$) in the same MC (in-distribution) and other MCs (out-of-distribution) using the test data that contains unseen model architectures to analyze how robust they are with regard to model architecture, embedded malware, model size, model task, and attack severity. More specifically, we aim

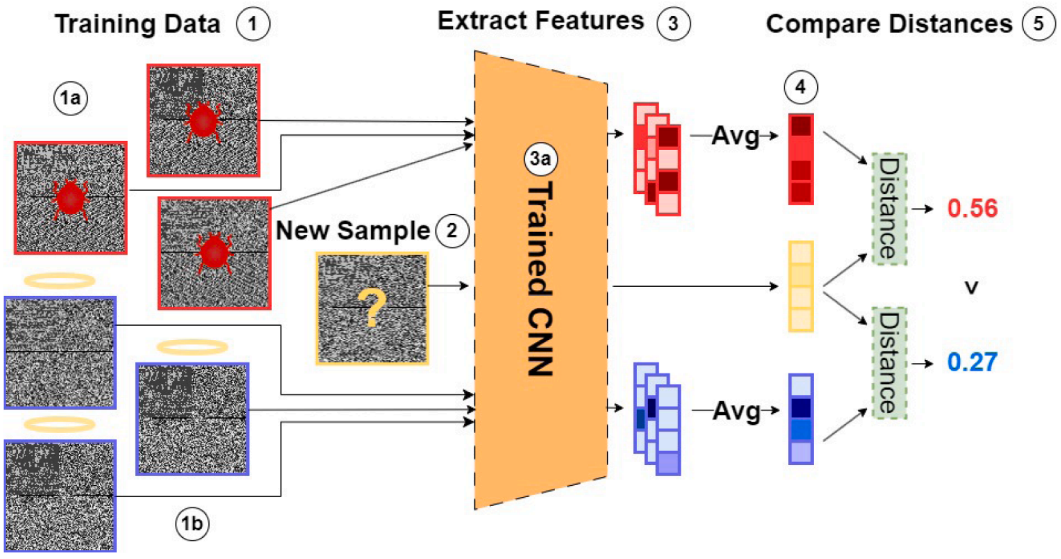


Fig. 3. Illustration of classifying a new sample (colored yellow, see (2)) using the trained CNN (3a) with the centroid method. We compute the embeddings of our benign and malicious training samples (colored blue and red, see (1a) and (1b)) and average them to get the centroid embeddings (4). Then, we classify the new sample by measuring the l_2 distance of its embedding to both centroid embeddings and giving it the label for the closer of the two (5). In this example, the new sample is labeled benign because it got 0.27 distance to the benign centroid as opposed to 0.56. This procedure is essentially the same as applying 1-Nearest-Neighbor w/l_2 distance using the centroids' embeddings. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

to show that the trained models can be used in real-life scenarios. All experiments were run on an NVIDIA GeForce RTX 3080 GPU, i9 intel core, and 32GB RAM.

4.1. Baselines

Research on detecting attacks on AI models is a relatively new field with few existing prior works [13,23,38]. Recent work [13] proposed detecting malicious model serialization attacks, which is outside the scope of our work. We compare against the remaining two prior works, additionally adapt MalConv [39] - the canonical raw-byte sequence approach to malware classification - to our setting as a learned baseline, and finally implement four simple threshold-based baselines that summarize a model by an aggregate per-parameter statistic. We label these seven baselines B1-B7 below and refer to them by these indices throughout the rest of the paper.

Shared XGBoost configuration.. Three configurations in this paper use XGBoost [49] on top of fixed-size feature vectors: B1 (Section 4.1.1), B2 (Section 4.1.1), and the *Ours - GF + XGBoost* ablation in Section 4.8.4. All three use the same classifier configuration: `tree_method="hist"` and `device="cuda"` (runtime parameters that select the histogram tree builder on GPU), with every other parameter at the xgboost-library defaults - 100 boosted trees, max depth 6, learning rate 0.3, full-row and full-column subsampling, ℓ_2 regularization $\lambda = 1$, and the binary-logistic objective. The three configurations therefore differ only in the input feature vector, not in the classifier itself, which keeps the comparison strictly between feature representations.

4.1.1. Baselines from prior academic work

B1 - Gilkarov et al. **[23].** Our prior work introduces a machine-learning steganalysis approach that trains supervised classifiers on small same-architecture CNN model zoos (the same zoo family later used in Section 4.4). It reported non-trivial detection accuracy from $X \geq 17$ but required on the order of 40,000 labeled training samples per zoo and was only validated on small same-architecture zoos. Our reproduction trains an XGBoost classifier (shared configuration in Section 4.1) directly on the flattened float32 weight vector of each model. We use B1 as the primary point of historical comparison against our FSL method in Experiment 1 (Section 4.8), since that is the setup in which the prior work first reported success.

B2 - Yin et al. **[38].** Yin et al. propose a steganalysis framework that targets three families of neural-network steganography: LSB encoding, correlated value encoding, and sign encoding. For each family they design a small fixed-size feature vector and train a logistic-regression classifier ([38], Eqs. (4)–(6)). For the LSB family - the attack family our paper is concerned with - Yin et al. extract four NIST randomness-test statistics from a single targeted mantissa bit plane: frequency (monobit), serial, approximate entropy, and cumulative sums (denoted $\phi_1, \phi_2, \phi_3, \phi_4$ in their paper). The resulting feature vector is therefore four-dimensional, and Yin et al. train and report results per single bit plane (e.g. Table 3 of [38] reports per-bit-plane detection accuracy on bit planes 14–18 of ResNet34, VGG16, EfficientNetB0, and MobileNet).

Yin et al. did not publish their code. Because our paper evaluates detectors across the full attack-severity range $X \in \{1, \ldots, 23\}$ rather than against a single bit plane, our reproduction extends Yin's design in two ways: we compute the same four $\phi_1 - \phi_4$ statistics but on each of the 23 mantissa bit positions, concatenating the per-position outputs into a 92-dimensional feature vector; and we feed this vector to an XGBoost classifier (shared configuration in Section 4.1) rather than logistic regression. The four statistics themselves are unchanged from Yin et al.; the per-position concatenation and the classifier choice are ours. As we show in Section 7.2 and Table 4, even our extracted-once-per-bit-position version of the feature extraction is roughly two orders of magnitude slower than ours at 10^8-parameter scale (173 s per model versus 0.49 s for our Grayscale-Fourpart representation). For this reason we do not include B2 in the main multi-architecture experiments (Experiments 1, 2, and 2.5), where running it across the full evaluation grid would be computationally prohibitive; instead, we compare against B2 in a dedicated experiment on a smaller homogeneous zoo (Section 4.8.4), where the runtime cost is manageable.

B3 - MalConv-lite (raw-byte 1D-CNN) [39]. MalConv [39] is the canonical end-to-end deep model for raw-byte malware classification: a learned 256-vocabulary byte embedding feeds a gated 1D convolutional network with global temporal max-pooling and a binary head,

trained directly on the bytes of an executable. We adapt this skeleton to our setting as a learned baseline that bypasses the image step entirely: the float32 weight tensor is read as a flat sequence of bytes and fed to a small MalConv-style 1D-CNN (8-d byte embedding, two parallel 1D convolutions of width 64 with stride 16, one ReLU and one sigmoid gate, global temporal max-pool, 64-d FC head, $\sim 1.5 \times 10^5$ parameters - hence "MalConv-lite"). For models larger than the GPU budget we take a deterministic 512 KB byte window at offset 0 of the flattened weight tensor, shared across all models so position semantics remain stable. The classifier is trained per attack severity X on the same training split as the FSL detector it is compared against, so the X-axis comparison is strictly matched in training data; we use AdamW (lr 10^{-3}, weight decay 10^{-4}), BCE loss, and early stopping on a held-out validation slice of the training data. The motivation for including B3 is to compare our image-based route against a state-of-the-art raw-byte sequence method - the canonical alternative paradigm for end-to-end deep detection of malicious binaries (cf. Sections 3.3 and 6). The empirical results across Experiments 1, 2, and 3 (Sections 4.8, 4.8.4) show that B3 succeeds only in the favorable regime of abundant training data combined with a structured payload (Experiment 1), and collapses to chance whenever either condition fails - supporting the design choices behind our specially crafted image representation of AI model weights.

Note on X-LSB-Attack-Fill usage. Both B1 [23] and the current study use the X-LSB-Attack-Fill variant exclusively. This ensures a fair comparison and eliminates confounding factors related to payload positioning. The baseline similarly employed payloads that fully utilize the embedding capacity, making our experimental comparison directly comparable in terms of attack characteristics.

4.1.2. Additional threshold-based baselines

To broaden the comparison and respond to reviewer requests for baselines beyond the two academic ones, we additionally implement four simple threshold-based baselines that operate on aggregate per-model statistics. They require only a benign-reference fit (mean, variance, or histogram) and a threshold calibration on the training split, with no full classifier training. Each baseline summarizes the model under test by a fixed-size feature vector, computes a scalar anomaly score as a distance from the benign-reference statistic, and classifies the model by thresholding that score. The threshold is calibrated on the training split to maximize training accuracy. All four are $O(n)$ in the number of model parameters with small constants, comparable in feature-extraction cost to our Grayscale-Fourpart representation (Section 3.3). Each baseline is a direct adaptation of a long-established family of single-feature detectors used in malware analysis or classical steganalysis, which we cite below.

B4 - Byte autocorrelation. For each model we compute the lag-1 Pearson autocorrelation of the least-significant-byte (lower-mantissa) sequence of the float32 weights in their natural parameter ordering. The intuition is that benign trained weights have nontrivial spatial correlation between adjacent parameters' LSBs, whereas LSB-substitution-fill attacks replace those bytes with payload content uncorrelated with the weight ordering. Lag-k autocorrelation is part of the classical family of randomness tests [50,51] routinely used to distinguish structured byte sequences from pseudo-random ones. The per-model feature is a single scalar and the anomaly score is L_2 distance to the benign-reference mean.

B5 - Byte entropy. We split each float32 weight into four bytes (sign + upper exponent, exponent low bit + upper mantissa, middle mantissa, lower mantissa) and compute the Shannon entropy of each byte position's histogram across all weights in the model. The per-model feature is a four-dimensional entropy vector and the anomaly score is L_2 distance to the benign-reference mean. Byte-entropy is a long-standing first-line tool for detecting encrypted or packed malicious binaries [52]; here we adapt it to per-byte-position statistics on float32 weight tensors.

B6 - Histogram KL-divergence. For each model we compute a 256-bin histogram of the least-significant-byte (lower-mantissa) values across all weights, with Laplace (add-one) smoothing. The per-model feature is a 256-dimensional probability vector. The anomaly score is the Kullback-Leibler divergence between the test distribution and the average benign-reference histogram from the training split. Distribution divergence between a test distribution and a benign reference is the canonical information-theoretic notion of steganographic detectability [53].

B7 - Weight-value distribution. We summarize each model's float32 weight values directly by mean, standard deviation, skewness, kurtosis, minimum, maximum, the fraction of weights in the near-zero band $[-0.01, 0.01]$, and a 100-bin histogram over $[-2, 2]$ with overflow clipped. The per-model feature is a 107-dimensional vector and the anomaly score is the standardized L_2 distance to the benign-reference mean. Moments-and-histogram features over the data values themselves are a classical family of steganalysis features [54,55], here adapted to numeric weight values rather than image-derived coefficients.

Table 1
Training-time hyperparameters of the Siamese-FSL feature extractors. Values shared across both architectures appear once; values that differ are split into the per-architecture rows.

Hyperparameter	OSL CNN	SRNet
Shared		
Loss	Triplet, ℓ_2 distance, margin 0.5	
Embedding	ℓ_2-normalized	
Optimizer	Adam, lr 6×10^{-5}, $(\beta_1, \beta_2) = (0.9, 0.999)$	
Image normalization	pixel / 255, no spatial augmentation	
Training modes	ES (1 ep.) / ST (5 ep.) / UB ($\leq$ 100 ep., loss-window stop)	
Test-time evaluation	Centroid (ℓ_2) and k-NN with $k = 1$	
Per-architecture		
Input shape	$100 \times 100 \times 1$	$256 \times 256 \times 1$
Mini-batch size	16	32
Dropout (conv blocks, dense)	0.5	per [37]
ℓ_2 kernel reg. (conv / dense)	2×10^{-4} / 10^{-3}	per [37]

4.2. Chosen feature extraction CNNs

We use two CNN architectures as feature extraction for our FSL models (See Section 3.4):

OSL CNN: We utilize the well-established CNN architecture designed for OSL introduced in [35] with images resized to 100x100. We denote this architecture as **OSL CNN**.

SRNet: Several works develop CNNs focused on image steganalysis. We utilize a prominent residual CNN called SRNet [37] in our experiments with images resized to 256x256. We denote this architecture as **SRNet**.

4.3. Training setup and hyperparameters

The training methodology is described in Section 3.4. Here we report the concrete numeric hyperparameters used in all experiments below; they are consolidated in Table 1.

The triplet loss is parameterized with ℓ_2 distance and margin 0.5. The CNN feature extractor's output is ℓ_2-normalized before the triplet distance is computed (Section 3.4). We optimize with Adam [47] at learning rate 6×10^{-5} and default $(\beta_1, \beta_2) = (0.9, 0.999)$. Two parameters differ between the two feature-extraction architectures (Section 4.2): the input image shape (matching the native input scale of each architecture's source paper, 100×100 for OSL CNN [35] and 256×256 for SRNet [37]) and the mini-batch size (the largest power of two that fits in our 10 GB GPU budget at the corresponding input scale, 16 for OSL CNN and 32 for SRNet). Regularization for the OSL CNN feature extractor is dropout 0.5 on the output of every convolutional block and on the penultimate dense layer, plus ℓ_2 kernel regularization with coefficient 2×10^{-4} on the convolutional kernels and 10^{-3} on the final 4096-d dense layer; SRNet is regularized as in the original publication [37].

Inputs are normalized by dividing pixel values by 255 to land in $[0, 1]$. We deliberately apply no spatial data augmentation (no flips, crops, or rotations), because the GF representation is parameter-position-stable by construction (Section 3.3); spatial augmentations would relocate pixels relative to their underlying weight indices and therefore destroy the position-stability that the CNN's spatial inductive bias relies on. For the FSL accuracy plots in Section 4.8, repeated-training run counts used to obtain the 95% confidence bands are reported per-experiment in the corresponding figure captions (e.g. ten runs for Experiment 1, thirty seeded train-split rotations for Experiment 2 AL).

4.4. Datasets

We create 4 different datasets for training and testing. We aim to comprehensively analyze different model architectures, sizes, etc. In addition, we reproduce results that use a spread-spectrum attack in the AI models [14] as a special study case on how the solution can handle unknown steganography attacks. We used the Grayscale-Fourpart image representation (Section 3.3) with image sizes 100 and 256 for all float32 datasets.

Attack-replication protocol.. Each dataset is constructed from a fixed pool of n_b benign model files. For every benign model and every attack severity $X \in \{1, \ldots, 23\}$, we apply X-LSB-Attack-Fill once with the dataset's malware payload to obtain a single attacked counterpart, yielding $23 \times n_b$ attacked models per dataset and $24 \times n_b$ models in total (the benign reference at $X = 0$ is the original pretrained or zoo model). The exception is the MaleficNet OOD dataset (Section 4.4.4), which is constructed by re-running the spread-spectrum attack of [14] per (architecture, malware payload) pair rather than per X. Validation accuracy of the benign and attacked models on the corresponding ground-truth task is reported in Appendix B; in particular, X-LSB-Fill at $X \leq 16$ leaves model accuracy effectively unchanged on every CNN architecture tested, which is the regime an attacker would realistically use and the regime our detector targets.

4.4.1. Small CNN zoos

Following the baseline work, we compile a dataset from a model zoo of small CNN models (≈2k float32 parameters per model) of the same architecture trained on STL10 [56]. We denote this dataset as **SCZ**. The model zoo has $n_b \approx 50{,}000$ benign models, all of the same fixed architecture (homogeneous zoo). We used the malware sample with sha256 6054f328f8d54d0a54f5e3b90cff020e139105eb5aa5a3be52c29dbea6289c30 on this dataset (denoted m_{6054f}). Since this dataset is used for comparison with the baseline [23] (see 4.1), we use a train/test split on models from the same model zoo in contrast with our methodology. We use 3 benign and 3 malicious models in the training split and the rest for testing.

4.4.2. Famous CNNs (small)

We compile a heterogeneous MC of $n_b = 7$ benign models drawn from different well-known CNN architectures of size at most 10M float32 parameters; per-architecture parameter counts span ≈2.5M (MobileNetV3Small) to ≈8M (DenseNet121). We use the ImageNet-pretrained models supplied in the Keras library [57]. The chosen architectures are: DenseNet121 [58], EfficientNetV2B0 [59], MobileNet (V1, V2, V3Small, V3Large) [60–62], and NASNetMobile [63]. We used the malware sample with sha256 77e05b52f51cfc8ec31f0dc2e544dc21b94250f35a5a353fd5e4e271e75bc45d on this model collection (denoted m_{77e05}). We chose the MZs MobileNet, NASNetMobile, and MobileNetV3Large for training and the rest for testing.

4.4.3. Famous CNNs (large)

We compile a heterogeneous MC of $n_b = 16$ benign models drawn from well-known CNN architectures of size between 10M and ≈100M float32 parameters; per-architecture parameter counts span ≈10M (EfficientNetV2B2) to ≈89M (NASNetLarge). We use the ImageNet-pretrained models supplied in the Keras library [57]. The chosen architectures are:

- DenseNet (169, 201) [58]
- EfficientNetV2 (B2,B3, S, M) [59]
- Inception (V3, ResNetV2) [64,65]
- NASNetLarge [63]
- ResNet (50(V1, V2), 101(V1, V2), 152(V1, V2)) [66,67]
- Xception [68]

For this MC, we used 3 concatenated malware samples. Their MD5 hashes are:

- 9de9993c77412ba8fa3200714dcd58f6
- edd73e79cb7b8abe0f569844931f4fdb
- 6d5c304bef8b2c32e904946f03b601d9

Denote this malicious payload $m_{9d_ed_6d}$. We chose the MZs DenseNet169, and NASNetLarge for training and the rest for testing.

4.4.4. Maleficnet OOD attack

We use the code from the authors in [14] to recreate spread-spectrum attacks. Using our methodology (Section 3), we create datasets from the benign and attacked models for additional evaluation. The recreated attacks (architecture + malware payload) are:

- DenseNet121 (Stuxnet, Destover)
- Resnet50 (Stuxnet, Destover, Asprox, Bladabindi)
- Resnet101 (Stuxnet, Destover, Asprox, Bladabindi, Cerber, Eq.Drug, Kovter)

The datasets we created above have 1 benign sample in each MZ and 2/4/7 malicious samples in the DenseNet121/Resnet50/Resnet101 architectures, respectively. Hence, a meaningful classification accuracy (better than weighted random guessing) will be more than 66%/80%/87.5%.

In the context of this paper, we call the dataset used in training an In-Distribution (ID) dataset, and other datasets not used in training are called Out-Of-Distribution (OOD). Studying the model performance on OOD data is essential to analyzing its generability.

4.5. Error bars

When applicable, we perform repeated experiments and show confidence interval plots using the seaborn python package [69]. The randomness in the trials stems from the CNN models' random initialization, and train sample shuffling.

4.6. Model evaluation metric

Evaluating steganalysis models for AI model steganography presents unique challenges compared to traditional binary classification tasks. We evaluate each trained model on different attacks severities, so we have multiple classification metric scores for each model; this makes it difficult to compare different models we trained. Moreover, performance on low embedding rates ($1 \leq X \leq 8$) is more significant than performance on high embedding rates ($8 < X$). For these reasons, we define and use a Weighted Metric (WM) that rewards models that are better at detecting the more evasive low ER attacks.

4.6.1. Rationale for the weighted metric

We define a custom Weighted Metric (WM) that reflects the practical importance of detecting low-embedding-rate attacks. Our design is motivated by two key considerations:

Prioritizing performance on Low ER: As demonstrated in Appendix B, attacks using fewer LSBs are substantially harder to detect. LSB steganography in the mantissa's least significant bits (e.g., $X \leq 8$, corresponding to embedding rates $\leq 25\%$) causes minimal perturbation to model weights and has negligible impact on model performance. These subtle attacks are precisely what sophisticated adversaries would employ to evade detection.

Practical threat modeling: From an adversary's perspective, attacks with embedding rates of 3%-25% (corresponding to $X = 1$ to $X = 8$ for float32) provide substantial payload capacity while maintaining stealth. For a 100MB model, even a 6% embedding rate allows embedding 6MB of malicious code - more than sufficient for most malware payloads. A detection method that only succeeds on high embedding rates ($\geq 50\%$) has limited practical utility, as such obvious attacks are unlikely to be deployed.

4.6.2. Weighted metric definition

The WM metric is a weighted accuracy of the model's result on some datasets on all s attack severities and benign samples. Let a_i the accuracy of a model on the dataset with i bits attacked, and let a_0 the accuracy of the same model on the benign dataset. The WM metric is given by:

$$WM = \frac{1}{2} \cdot (a_0 + \sum_{i=1}^{s} \frac{(s - i + 1)a_i}{\sum_{i=1}^{s} i})$$

The WM metric is bounded in the range [0,1] by definition.

4.7. Plot types

We use 2 main types of plots to showcase our results:

1. Only Model LSB (OML): These plots show classification accuracy of models trained on some attack severity $1 \leq \hat{X} \leq 23$ that are then tested on benign and attacked models that were attacked with attack severity $\hat{X}$. The x-axis shows Model LSB and the y-axis shows mean accuracy for both benign and attacked models. These plots help us get an initial sense of the success of a model.
2. All LSBs (AL): These plots show the weighted metric (see Section 4.6) of models trained on some attack severity $1 \leq \hat{X} \leq 23$. The x-axis shows Model LSB, and the y-axis shows the weighted metric. These plots help us decide what X value results in the best global-performing model on a specific dataset.

4.8. Results

4.8.1. Experiment 1 - SCZ baseline

In this experiment, we train classification models on the SCZ MC (Section 4.4.1) using the UB training strategy (see Section 3.4). The main goal of this experiment is to measure the success (test accuracy) of models trained using our proposed methodology (Section 3) versus B1 [23]. To this end, we closely replicate the training/testing procedure they used to make the comparison strict. B1 trained a model for each attack severity and each model architecture individually. Here, we focus on the STL10 model zoo and train our classification models on 3 STL10 models (benign and attacked) on each attack severity (23 different models). We repeat the training process 30 times and show mean and confidence interval results. See OML Fig. 4. The plot shows the #LSB in the training dataset each model learned versus the test accuracy on the STL10 MZ. The CNN model with centroid evaluation had very close results to B1 for $\#LSB \in [18, 23]$ (56%-71% ER). For $\#LSB \in [8, 18]$ attacks (25%-56% ER), it showed stable results in the confidence interval [0.85, 1], while B1 failed. The CNN model with 1NN evaluation also had good results but was less stable. This is likely due to the model being overfit. We successfully trained FSL models that outperformed B1 in terms of accuracy using only 6 models for training, as opposed to 40,000 models for B1.

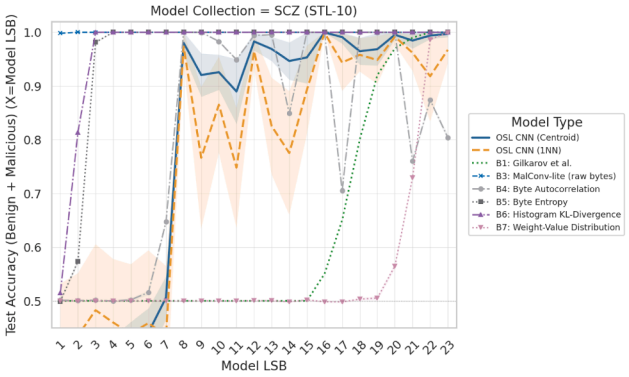


Fig. 4. Experiment 1 OML results (see Section 4.7). FSL models (OSL CNN, Centroid and 1NN) are trained on only 6 SCZ samples; B1 (Gilkarov et al.) is reproduced from [23] and plotted as a dotted green line. B3 (MalConv-lite, raw-byte 1D-CNN) and B4-B7 (Section 4.1.2) are given a Gilkarov-style training budget of 40,000 train / 10,000 test models on the SCZ STL10 zoo. Bands are 95% confidence intervals across runs (FSL) or seeded train/test rotations (B3-B7). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

The threshold-based baselines B4-B7 are also overlaid in Fig. 4, given the same Gilkarov-style training budget that B1 uses (40000 train / 10,000 test models, 10 random splits). On this homogeneous, real-malware-payload zoo *three* of the four baselines saturate to near-perfect accuracy from very low X: B6 (Histogram KL-Divergence) reaches $\approx$ 100% from $X = 3$ and B5 (Byte Entropy) from $X = 4$, dominating both our FSL method and B1 in this specific setting. B4 (Byte Autocorrelation) succeeds from $X \geq 8$, while B7 (Weight-Value Distribution) only catches up at $X \geq 21$, where the attack starts to perturb the value-domain statistics. The raw-byte-sequence baseline B3 (MalConv-lite), trained on the same 40,000 SCZ STL10 models per X, is also competitive in this regime: it reaches $\geq 99.9\%$ accuracy from $X = 1$, on par with the best threshold-based baselines once given enough labeled data and a payload with byte-level statistical structure. This is a striking contrast with the heterogeneous-zoo setting (Experiments 2 and 2.5 below) and with the random-payload setting (Experiment 3, Section 4.8.4), where B3-B7 collapse to near-random; the contrast is unpacked in our caveat in Section 4.8.4.

4.8.2. Experiment 2 - small famous CNNs

In this experiment, we train classification models on the small famous CNNs using the UB training strategy (see Section 3.4). The main goal of this experiment is to see how the techniques fare with the larger CNN architectures and whether the models succeed on unseen model architectures and unseen attacks. We train models on all 23 attack severities like before, but this time, we measure their performance on the other attacks (unseen during training) using the Weighted Metric introduced in Section 4.6. This metric gives us a general way to score models and rewards success on the more delicate attacks (smaller X value), which are a major challenge in this area. In addition to evaluating the models on the small famous CNNs test set, we also evaluate the models on the large famous CNNs (See Section 4.4). These architectures serve as an OOD test for the models trained on the small famous CNNs, with a size difference of (10M-100M). We show average results over 50 repeated trials and confidence intervals. We start by examining AL and OML results on the small CNNs test set.

From OML Fig. 5, we can see that SRNet models (centroid and NN evaluation) from $X \geq 6$ got perfect accuracy with great confidence, and OSL CNN models were close behind with accuracy in the confidence interval $[0.9, 1]$. These are substantial results with good accuracy for ERs 19%-100% as opposed to 50%-100% in B1. As we gather from Appendix B, attacks with ER 3%-50% largely don't affect model performance which means such attacks are likely to be used, and so we

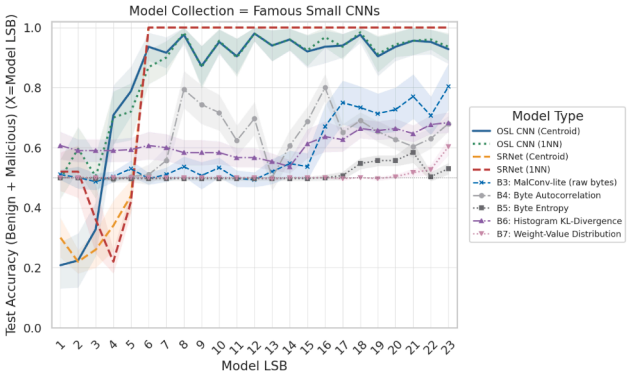


Fig. 5. Experiment 2 OML ID results. FSL models trained on the famous small CNNs train set and tested on the small CNNs test set. The raw-byte baseline B3 (MalConv-lite) and the threshold-based baselines B4-B7 (Sections 4.1.1, 4.1.2) are overlaid; their bands are 95% confidence intervals across 30 random rotations of the 3-of-8 architecture training split. B3 is given the same FSL training budget (3 train architectures, 6 samples per X) so the comparison reflects the data regime, not the architecture budget.

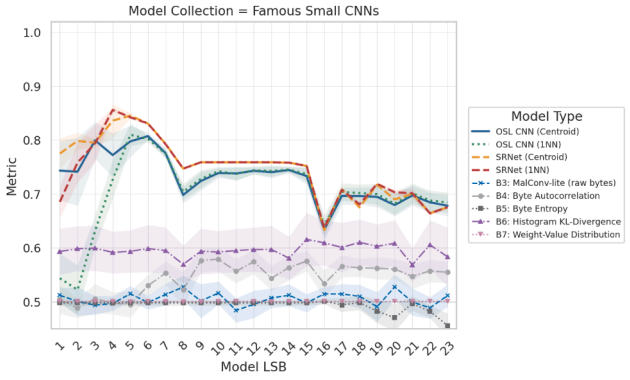


Fig. 6. Experiment 2 AL ID results. FSL models trained on the famous small CNNs train set and tested on the small CNNs test set. The raw-byte baseline B3 (MalConv-lite) and the threshold-based baselines B4-B7 (Sections 4.1.1, 4.1.2) are overlaid using the same 30 seeded train-split rotations and the paper's Weighted Metric formula with measured benign accuracy a_0.

need effective countermeasures for them. SRNet with 1NN evaluation had remarkable results in attacks with $X = 2$ (ER = 3.1%) with a confidence interval of approximately 0.55-0.65. OSL CNN models with centroid evaluation were the most successful for $X \in \{3, 4, 5\}$ with mean accuracy 0.6, 0.7, and 0.8 respectively with good confidence. The non-FSL baselines paint a sharply different picture in this heterogeneous-zoo setting: B5 (Byte Entropy) and B7 (Weight-Value Distribution) sit essentially at chance (≈ 0.5) across all X, while B4 (Byte Autocorrelation) and B6 (Histogram KL-Divergence) only manage to oscillate in the $0.55 - 0.7$ range and never approach FSL accuracy. The raw-byte-sequence baseline B3 (MalConv-lite), given the same 6-sample FSL training budget, is also at chance (≈ 0.5) for $X \leq 15$ and only climbs to $\approx 0.67 - 0.80$ at $X \geq 16$, well below the FSL detectors and consistent with the data-scale and position-semantics arguments of Section 6. The contrast against their near-perfect behavior on Experiment 1 (Fig. 4) is direct evidence that these baselines do not transfer beyond the same-architecture setting - a point we expand on in Section 4.8.4. After examining the models' performance on the attack severity they trained on, we move on to examine the results more globally on all attack severities.

From AL Fig. 6, we see models trained on subtle attacks had the highest success globally. The best weighted metric result is ~85% using SRNet on $X = 4$. This is an impressive result since the metric, by definition, requires adequate performance on both benign and malicious data. In particular, with a metric rating of 0.85, accuracy on benign data must be $\geq 70\%$. Models trained using our proposed methodology

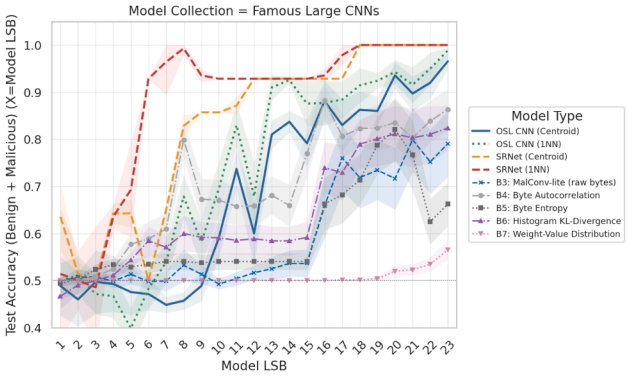


Fig. 7. Experiment 2 OML OOD results. FSL models trained on the famous small CNNs train set and tested on the large CNNs test set. B3 (MalConv-lite) and B4-B7 (Sections 4.1.1, 4.1.2) are overlaid; the same small-CNN training models that fit the FSL detector are also used as the training data for B3 and the benign reference for B4-B7.

achieve good results with very few training samples, and we prove their worth for all attack ranges as opposed to B1, which trained a separate model for each attack severity. The non-FSL baselines B3-B7 trail FSL across all anchor X in the WM dimension as well: B6 plateaus around 0.6, B4 around 0.55, and B5/B7 stay close to 0.5; the raw-byte baseline B3 (MalConv-lite) is similarly bounded below FSL at every anchor X. Every FSL configuration sits between 0.65 and 0.85 - a uniformly higher and more stable WM regardless of which X the FSL detector is anchored on.

Moving on, we look to evaluate the models on OOD data. From OML Fig. 7, which shows results on the large famous CNNs, we can see the models exhibit accuracy $\geq 70\%$ for $X \geq 6$ and $X \geq 9$ with the SRNet and OSL CNN models respectively. There's no noticeable difference between the centroid and 1NN evaluation. We note the SRNet models have perfect accuracy for $X \geq 16$ ($\geq 50\%$ ER) with great confidence. We feel these results are a good demonstration of the models' generality with regard to model architecture and embedded payload. The threshold-based baselines B4-B7 do begin to climb on the OOD large-CNN test set, but only at moderate X: B4 reaches ~ 0.8 from $X = 8$ and B6 climbs to ~ 0.8 at high X, while B5 and B7 remain near chance until $X \geq 16$. The raw-byte baseline B3 (MalConv-lite), trained on the same 6-sample FSL split, transfers no better OOD than ID: it stays at chance for $X \leq 15$ and only reaches $\approx 0.66 - 0.79$ at $X \geq 16$, mirroring its small-CNN ID behavior. Across the full X range FSL still strictly dominates, and the gap is largest precisely in the low-X regime that the Weighted Metric (Section 4.6) rewards.

4.8.3. *Experiment 2.5 - MaleficNet*

We take the models trained in Experiment 2 and evaluate them on the MaleficNet attacked models presented in Section 4.4. See Table 2. We display the Run Num where the model was created, the model LSB, evaluation type, weighted metric on ID data, and mean accuracy on maleficnet data. We filter models which had $\geq 60\%$ accuracy on the benign and malicious maleficnet models with 1NN evaluation or with centroid evaluation. Out of these, we show the top 3 with centroid and the top 3 with 1NN evaluations sorted by the Weighted Metric on the ID small famous CNNs test set.

We first note that only OSL CNN models had success on MaleficNet data, this shows an added benefit of the OSL model over the SRNet model which had better performance on ID data.

Why OSL CNN transfers and SRNet does not. The two architectures embody near-opposite assumptions about the test distribution, and the ID/OOD ranking inversion (SRNet wins ID in Figs. 5 and 7; only OSL CNN transfers in Table 2) is the empirical signature of that difference.

Training objective. OSL CNN [35] is a Siamese verification network trained with a same/different cross-entropy on Omniglot pairs; the em-

Table 2
Classification accuracy of selected models from Experiment 2 on MaleficNet OOD attack.

Run Num	Model LSB	Eval Type	WM	OOD Acc
48	1	C	**87**	**80**
37	2	C	82	71
31	2	C	81	80
13	1	NN	80	64
38	1	NN	73	67
42	3	NN	64	79

bedding is forced to separate arbitrary class boundaries rather than commit to any specific signature. Koch et al. [35], Section 4.4, report the property that matters here directly: a network trained only on Omniglot reaches 70.3% on a 10-way one-shot MNIST task with no MNIST training (vs. 26.5% for 1-NN). Our X-LSB-Fill → MaleficNet transfer is the same kind of cross-distribution reuse of a verification embedding. SRNet [37], by contrast, is trained end-to-end as a binary classifier on cover/stego pairs of a *single* embedding scheme at a time, with all filters randomly initialized and learned so as to "adapt to a greater variety of stego signals because the polarity of and dependencies among embedding changes vary significantly across different steganographic methods" [37]. The architecture is therefore optimized to amplify whichever residual best discriminates *this* embedding from cover.

Evidence from the source papers. Boroumand et al. [37], Table II, quantify the price of that specialization: even when training and test attacks both belong to the spatial-domain LSB-cost family (WOW, HILL, S-UNIWARD, MiPOD), swapping the train and test algorithm costs roughly 10–20 P_E points. Our setting demands a more aggressive transfer - from mantissa-LSB byte-level perturbation (X-LSB-Fill) to LDPC-coded spread-spectrum modulation (MaleficNet) - so the same architectural prior that pays off ID becomes a specialization tax OOD.

Image-representation × signal interaction. X-LSB-Fill produces a localized, high-frequency mantissa-byte fingerprint on the GF image, which SRNet's noise-residual front segment - in which pooling is deliberately disabled to "avoid decreasing the energy of the stego signal" [37] - is purpose-built to extract; this is consistent with its stronger ID accuracy. MaleficNet's spread-spectrum payload is a low-amplitude, broadband perturbation distributed across the magnitudes of many parameters via LDPC encoding, with no reason to align with the high-pass kernel basis SRNet learned for X-LSB-Fill. OSL CNN's generic 4-conv stack [35] carries no such band-specific prior, so any cue separating benign from attacked GF images is admissible into the verification embedding. The top entry in Table 2 (OSL CNN, centroid, X=1, WM 87 / acc 80) is therefore the expected outcome of these design choices rather than an isolated result.

We therefore conclude the OSL CNN architecture is more useful when developing more generic steganalysis networks, to work across a large variety of attacks, models, etc.

We see that all models that had success were trained on $X \in \{1, 2, 3\}$, this might be due to the model being fit to detect very delicate changes. The top model (centroid evaluation with $X = 1$) in the table exhibited a weighted metric of 87% and 80% accuracy on the MaleficNet data, this shows that indeed, some models trained on our unrelated LSB substitution attacks can detect very different spread-spectrum attacks, and simultaneously not compromise ID performance which is substantial. It's important to note that the WM value of 87% doesn't lie within the confidence interval seen in Fig. 6 which is approximately (0.75, 0.8) for the OSL CNN models with centroid evaluation. Nevertheless, this is a motivating result. It's also worth mentioning that the MaleficNet models are of DenseNet121, ResNet50, and ResNet101 architectures, of which none are in the small famous CNNs train set (see Section 4.4). In conclusion, this experiment further outlines the generality of trained models using our methodology, in particular, regarding attack type, embedded payload, and model architecture.

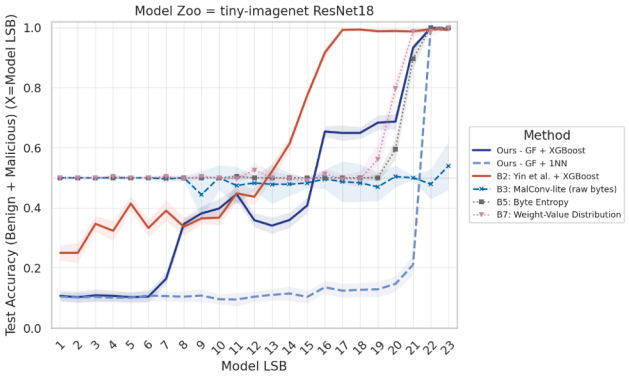


Fig. 8. Test accuracy of six steganalysis methods on the ResNet18-TinyImageNet model zoo [56] (5-fold × 5-repeat stratified CV, X = Model LSB, random-bit payloads). Methods: *Ours - GF + XGBoost* and *Ours - GF + 1NN* (two classifiers on our Grayscale-Fourpart pixels), *B2* (Yin et al. + XGBoost), *B3* (MalConv-lite, raw-byte 1D-CNN), *B5* (Byte Entropy), *B7* (Weight-Value Distribution). All baselines are defined in Section 4.1.

4.8.4. Experiment 3 - Homogeneous ResNet18 zoo comparison against B2

In this experiment, we compare our method against B2 (Yin et al. [38]) and several other baselines on a homogeneous model zoo. In their original setup, Yin et al. train one detector per single targeted mantissa bit plane on a balanced training set of 100 benign and 100 malicious models with controlled payloads in $\{0.05, 0.4, 0.8, 1.0\}$ bits per parameter, and report per-bit-plane detection accuracy on bit planes 14–18 of ResNet34, VGG16, EfficientNetB0, and MobileNet ([38], Table 3). The reported accuracies on ResNet34/CIFAR10 range from 100% in the most favorable settings (the highest two bit planes, payload ≥ 0.4) down to 65–87% in the most subtle settings (lower bit planes 17–18 or low payload 0.05). We note that Yin et al.'s axis is *which single bit plane is targeted*, while our paper's axis is *how many of the lowest mantissa bits are overwritten* via X-LSB-Attack-Fill - these are not directly comparable, which motivates re-evaluating B2 from scratch on our datasets here. The complementary deployment-cost comparison against B2 is reported in Section 7.

We compare detection accuracy on the ResNet18-TinyImageNet model zoo [56] (5-fold × 5-repeat stratified CV, random-bit payloads) against six competitor configurations, all evaluated under matched protocol so the comparison is strictly apples-to-apples. The competitors are: B2 (Yin et al., trained as XGBoost on the 92-dimensional NIST features); B3 (MalConv-lite, raw-byte 1D-CNN on a 512 KB byte window); B5 (Byte Entropy) and B7 (Weight-Value Distribution) - two non-learned naive baselines; and two classifier ablations of our own representation - *Ours - GF + XGBoost* (XGBoost trained on the Grayscale-Fourpart image pixels, shared configuration in Section 4.1) and *Ours - GF + 1NN* (1-nearest-neighbor on GF pixels). The two GF-based learners share our representation but use different classifiers, isolating representation choice from classifier choice. The full definitions of B2, B3, B5, and B7 are given in Section 4.1; B4 (Byte Autocorrelation) and B6 (Histogram KL-Divergence) are omitted from Fig. 8 because, on this random-payload setting, both of them stay at exactly chance accuracy (≈ 0.5) for every $X \in [1, 23]$ - B4 because the attacker's pseudo-random bytes carry the same lag-1 autocorrelation (≈ 0) as the already near-uniform benign mantissa bytes, and B6 because the attacked byte histogram is statistically indistinguishable from the benign one when the payload itself is uniform. Neither baseline therefore has anything informative to plot here. We do report both B4 and B6 in the other experiments above (Section 4.8), where the malware-payload attacks they are designed to detect produce a non-trivial signal.

See Fig. 8 for the detection accuracy of all six methods across attack severities $X \in [1, 23]$. We observe three regimes. In the *high-severity regime* ($X \geq 20$), the GF-based learners and B2 approach perfect classification: on this homogeneous zoo of 116 identically-architected ResNet18

models, the tight benign reference makes even trivial byte- or value-level tests sufficient once the attack fully corrupts byte boundaries of the mantissa. In the *mid-severity regime* ($8 \le X \le 19$) the methods separate: B2 is the most consistent learner across this range (climbing from 34% at $X = 8$ to > 99% at $X = 17$), our GF + XGBoost reaches ∼65-70% in the $X = 16$-19 region, GF + 1NN flatlines near chance until $X = 21$, and the non-learned baselines B5 and B7 hover at exactly 50% throughout. The raw-byte-sequence baseline B3 (MalConv-lite) is, strikingly, the *worst* learner across the entire range: despite the architecture-matched zoo, it never exceeds ≈ 55% test accuracy at any X even at $X = 23$, while its train accuracy climbs above 90% from $X \ge 9$. This is overfitting in its purest form: ∼93 training samples is far below MalConv's natural training regime ($4 \times 10^5 - 2 \times 10^6$ binaries [39]), and a uniform random LSB payload provides no byte-level statistical structure for a 1D-CNN to exploit. Finally, in the *low-severity regime* ($X \le 7$), *all six methods* fail to distinguish benign from attacked models, consistent with the intrinsic difficulty of detecting attacks that modify only part of the least-significant mantissa byte.

Caveat: when do naive baselines work? The reader will note that the same naive baselines, when evaluated on the SCZ-STL10 zoo of Experiment 1 (Section 4.8, Fig. 4), *exceed* both our FSL method and B1 (Gilkarov et al.) - B6 (Histogram KL-Divergence) and B5 (Byte Entropy) reach ≈ 100% accuracy from $X = 3$ and $X = 4$ respectively. Yet the same baselines, when evaluated on the famous-CNN zoos of Experiment 2 (Figs. 5 and 7), *collapse* to near-random accuracy across all X. This apparent inconsistency is reconciled by noting that the non-FSL baselines (B3-B7) depend on three conjoint conditions: (i) *architectural homogeneity*, so the benign reference distribution is tight enough that small attack-induced shifts are statistically detectable; (ii) *a payload with byte-level statistical structure* (real malware bytes), so the attacked-byte distribution is meaningfully different from the already-near-uniform benign mantissa - on the same ResNet18 zoo with random-bit payloads (Fig. 8), B5 and B7 stay at exactly 50% until $X \ge 20$; and (iii) *abundant training data* for stable threshold calibration or sequence learning - with only 3 benign and 3 attacked training samples (the FSL budget) on the heterogeneous famous-CNN zoos, threshold-calibration variance dominates the attack signal for B4-B7 at low X, and the same data scarcity prevents B3 (MalConv-lite) from learning a transferable byte-level attack signature. None of these conditions can be assumed in realistic deployment, where models come from diverse architectures, an adversary may use encrypted or otherwise high-entropy payloads, and labeled attacked samples are scarce by definition. Our few-shot CNN approach is robust to all three conditions failing: the OSL CNN and SRNet feature extractors, trained on only 6 models (Section 3.4), maintain non-trivial detection accuracy across both heterogeneous-zoo and structured-payload settings (Figs. 5–7). We emphasize that our method also retains the efficiency advantage over B2 shown in Section 7.2.

5. Limitations and future work

Our research focused on detecting LSB substitution steganography attacks and introduced new steganalysis methods. While experimental results show promising results in LSB attacks and spread-spectrum attacks, future research should cover evasive attack vectors like maleficnet [14] directly. This can be done by constructing a more comprehensive dataset that includes models attacked using MaleficNet, and training and evaluating the methods on it. Moreover, covering MaleficNet might require adjusting the feature extraction methodology to include new features that help detect the spread-spectrum attacks, which produce different changes in the data compared with LSB steganography.

This work introduces a general methodology and software framework for research [40], however, we focus on CNN models with float32 weights. Extending the methodology to lower-precision data types - such as the 8-bit, 4-bit, and 2-bit quantization regimes increasingly used by LLMs - would require designing a new image representation tailored to each format's bit layout, and constitutes future work.

Network-scale coverage.. A possible reading of our experimental scope is that the method is only validated on small networks. The actual parameter range covered by the experiments is ∼2k (Small CNN Zoos, Section 4.4.1) to ∼89M (NASNetLarge in Famous CNNs Large, Section 4.4), spanning approximately 24 distinct CNN architectures including DenseNet, EfficientNetV2, Inception, MobileNet, NASNet, ResNet, and Xception. The runtime/memory study in Section 7.2 (Tables 4 and 5) further measures the cost of our feature extraction up to 10^8 parameters and shows that it scales linearly in the parameter count n, with measured constants ≈ 0.49 s on a single core and ∼1.77 GiB peak memory at $n = 10^8$. Extrapolating the feature-extraction stage to LLM scale ($n \ge 10^9$) is therefore a linear-cost extension and does not require redesigning the image representation. What does require additional work for LLM scale is the FSL training and evaluation pipeline: the current training set consists of as few as 6 benign + attacked CNN model files (Section 4.3), and validating the method on attacked LLMs would require an analogous attacked-LLM model zoo, which to our knowledge is not yet publicly available. The bottleneck for scaling the method to LLMs is therefore data availability, not algorithmic reach.

Future direction: tile-and-aggregate scaling.. A natural way to apply the FSL detector to networks substantially larger than the ∼10^8-parameter regime studied here, without retraining the detector at each target model size, is a *tile-and-aggregate* approach. The detector is trained once at its native input resolution ($Z \times Z$ pixels, here $Z = 100$ for OSL CNN and $Z = 256$ for SRNet); at inference time on an n-parameter target model, the full-resolution Grayscale-Fourpart image is partitioned into $\lceil 4n/Z^2 \rceil$ disjoint $Z \times Z$ tiles, each tile is classified independently with the trained detector, and the per-tile embeddings or scores are aggregated into a single per-model decision (e.g. max-pooling for any-tile-suspicious detection, mean-pooling for a global score, or attention-weighted pooling). The Grayscale-Fourpart representation is parameter-position-stable by construction (Section 3.3), so within each tile a pixel still maps to a fixed weight role; the spatial inductive bias that makes the FSL detector effective at the tile level is preserved. The open empirical question is whether a low-embedding-rate X-LSB-Fill attack carries enough discriminative signal at the tile level for an aggregation rule to recover the model-level decision, and which aggregation rule is optimal under which embedding-rate regime. We leave this study to future work.

6. Related work

In this section, we review relevant literature across three key areas: steganographic attacks on AI model weights, mitigation techniques for these attacks, and the use of image-based representations for classification tasks.

6.1. AI model weights steganography

The use of AI models as covert channels for malware delivery has emerged as a significant security threat. Several works have demonstrated the feasibility of embedding malicious payloads within neural network weights using steganographic techniques.

StegoNet [12] introduced the concept of using deep neural networks as stegomalware, demonstrating that model weights can serve as effective carriers for hiding malicious code. The authors showed that large model sizes provide substantial hiding capacity while maintaining model performance, making detection challenging for traditional security tools.

EvilModel [10,11] demonstrated practical attacks that embed malware within neural network model files. Wang et al. showed that attackers can leverage the redundancy in floating-point model parameters to hide malicious payloads that can be extracted and executed on target systems. Their EvilModel 2.0 work further refined these techniques and demonstrated resilience against basic detection methods.

MaleficNet [14] introduced sophisticated spread-spectrum steganography techniques for AI models using low-density parity-check codes. This work demonstrated that steganographic attacks can be made resilient to common model modifications such as adding random noise, parameter pruning and fine-tuning, significantly complicating previous mitigation efforts that relied on these methods [24,25].

Building on these attack methodologies, our prior work [23] introduced the **X-LSB-Attack** framework, which systematically embeds malware in the least significant bits of model weight mantissas. This work established a comprehensive methodology for simulating LSB-based steganography attacks across different model architectures and demonstrated that such attacks can maintain high stealth while providing substantial payload capacity. The X-LSB-Attack can be seen as a generalization of the method used by previous works such as StegoNet [12], and EvilModel [10]; the fill variant is used to create a more standardized attack surface. The baseline steganalysis work in [23] used the fill variant, so in order to build upon the previous analysis and results, we do so too.

6.2. AI model steganography mitigation

Mitigation strategies for AI model steganography fall into two primary categories: detection-based steganalysis and zero-trust content disarm and reconstruction (CDR) approaches.

6.2.1. Detection-based steganalysis

Statistical-based detection. Yin et al. [38] proposed one of the first steganalysis methods for neural networks based on parameter statistical bias. Their approach extracts statistical features from model weights - including moments, entropy measures, and distribution characteristics, and uses machine learning classifiers to distinguish between benign and attacked models. While achieving promising accuracy on controlled datasets, their method faces significant computational scalability challenges. As we demonstrate in Section 7.2, the feature extraction process becomes prohibitively expensive for models with hundreds of millions of parameters, limiting practical deployment.

Machine learning-based detection. Our prior work [23] introduced the first machine learning approach to AI model steganalysis, training supervised models to detect LSB steganography attacks. However, this method required approximately 40,000 training samples and demonstrated effectiveness primarily on high embedding rates (50%-100%). The current work addresses these limitations by leveraging few-shot learning techniques to dramatically reduce data requirements while improving detection of subtle attacks.

6.2.2. Zero-trust mitigation approaches

As an alternative to detection, several works have explored preventive approaches that modify models to disrupt potential steganographic payloads.

Parameter modification techniques. Dubin [24] proposed methods for disarming attacks inside neural network models through strategic parameter modifications. These techniques include quantization, random bit flipping, and controlled noise injection designed to corrupt hidden payloads while preserving model functionality. While effective against basic LSB attacks, these approaches may damage model performance and can be circumvented by sophisticated encoding schemes with error correction such as MaleficNet [14]. Amit et al. [25] similarly explored using adding random noise to the parameters to mitigate steganography. However, sophisticated attacks using error-correcting codes [14] can resist such modifications as well.

The primary limitation of zero-trust CDR approaches is their potential to alter model behavior, which may be unacceptable for applications requiring certified or validated models. Detection-based approaches, as we pursue in this work, offer the advantage of identifying threats without modifying the original model, enabling users and repositories to filter malicious content while preserving benign models intact.

6.3. Image-based representations for classification

The use of image representations to transform non-image data for classification with computer vision techniques has proven successful across multiple domains, particularly in malware analysis.

Malware binary visualization. Nataraj et al. [70] pioneered the approach of visualizing malware binaries as grayscale images, demonstrating that malware families exhibit distinctive visual patterns when their executable code is rendered as images. This seminal work has inspired extensive research in image-based malware classification [71,72].

Recent works have applied deep learning to malware images with considerable success. Kalash et al. [72] developed the M-CNN architecture based on VGG-16, achieving high accuracy on malware family classification. Multiple studies [73–75] have demonstrated that transfer learning from ImageNet-pretrained models (VGG-16, ResNet50, Inception V3, EfficientNet) can effectively classify malware when executable binaries are converted to RGB or grayscale images.

Few-shot learning with image representations. Most relevant to our approach, Hsiao et al. [36] applied one-shot learning with Siamese neural networks to malware image classification. They demonstrated that converting malware binaries to grayscale images and training Siamese CNNs enables effective classification with minimal training samples, addressing the challenge of rapidly emerging malware variants. Our work extends this paradigm to a fundamentally different domain: rather than detecting malware executables through their binary patterns, we detect malware hidden within AI model weights through steganographic embedding.

Network traffic visualization. Beyond malware binaries, image-based approaches have been applied to network security. Recent work [76] has explored converting network packet sequences to images for intrusion detection and malicious traffic classification using CNNs.

Raw-byte sequence approaches. A complementary line of work skips the image step entirely and treats a binary as a flat sequence of bytes. The canonical example is MalConv [39], which feeds the raw bytes of a Windows PE file into a 1D convolutional network with a learned byte embedding and detects malicious executables end-to-end. MalConv is well-matched to its problem: detecting localized malicious *behavior* (function bodies, suspicious imports, packer signatures) inside a $\sim 2 \times 10^6$-byte stream where byte-level meaning is highly context-dependent and the file layout can be arbitrarily rearranged [39]. Our problem differs along three axes that make this paradigm a poor fit. (i) *Detection signal:* X-LSB-Attack-Fill produces a globally distributed statistical perturbation across the mantissa region of every parameter, rather than a localized code feature; MalConv's strength at locating rare 500-byte features in a long stream is therefore architecturally mismatched. (ii) *Position semantics:* for models drawn from the same architecture, parameter index N refers to a comparable weight, so a 2D image with stable pixel-to-parameter alignment is a meaningful spatial signal. Position stability is load-bearing even when the underlying attack is uniform (axis i above): different layers and float32 byte-regions have different baseline weight distributions, so the same bit-flip produces a statistically different fingerprint depending on where in the image it lands; a CNN with stable pixel-to-parameter alignment can learn these region-specific signatures jointly, whereas a representation without position stability would have to average them away. The argument applies a fortiori to non-uniform variants such as partial-ER or layer-targeted attacks, where the signal is genuinely localized - whereas MalConv must explicitly avoid relying on absolute byte position because EXE contents can be reordered without changing meaning [39]. (iii) *Data scale:* MalConv is trained on $4 \times 10^5 - 2 \times 10^6$ labeled binaries to learn its byte embedding from scratch [39], whereas our setting trains on 6 models (Section 4). The image route lets us reuse pretrained vision backbones and metric-learning losses, which is what makes few-shot detection feasible. We therefore see MalConv as orthogonal rather than competing: a strong baseline for behavioral malware classification on EXE files, but addressing a different threat model than steganalysis of numeric weights.

Table 3
Image-based malware classification methods vs. our image-based steganalysis of model weights. Prior works convert executable bytes into images and detect malicious *behavior* encoded in compiled instructions (predominantly Windows PE corpora; Xiao et al. further requires PE-specific section parsing); our method converts float32 model weights into images and detects steganographic *modification* of mantissa bits. **bytecode → gs**: standard byte-to-grayscale rendering of the executable byte stream. The *Detection signal* column reports the visual feature each method exploits, with the classifier in parentheses.

Method	Target medium	Image construction	Detection signal (classifier)	Few-shot
Nataraj et al. [70]	Executable bytes	bytecode → gs	Bytecode texture, per-family (GIST + k-NN)	No
Gibert et al. [71]	Executable bytes	bytecode → gs	Bytecode texture (end-to-end CNN)	No
Kalash et al. [72]	Executable bytes	bytecode → gs	Bytecode texture (VGG-16 transfer)	No
Vasan et al. [73]	Executable bytes	bytecode → RGB	Bytecode texture (CNN ensemble)	No
Chaganti et al. [75]	Executable bytes	bytecode → RGB	Bytecode texture (EfficientNet transfer)	No
Xiao et al. [77]	PE bytes + section table	bytecode → gs + section overlay	Bytecode texture + PE section layout (VGG-16 + SVM)	No
Hsiao et al. [36]	Executable bytes	bytecode → gs	Bytecode texture (Siamese metric)	Yes
Ours (Grayscale-Fourpart)	**Float32 model weights**	**4 × 8-bit mantissa-byte split, 2 × 2 tile**	**Mantissa-LSB statistical perturbation (OSL CNN / SRNet metric)**	**Yes**

Distinction from our work. Table 3 summarizes the comparison; the design choices being situated here are developed in our design rationale (Section 3.3.1), and this paragraph places those choices in the prior-art landscape. The image-based malware classification line of work (rows 1–7) treats the input as an executable byte stream, and the signal that survives the byte-to-pixel rendering is *behavioral*: it is the visual texture of compiled instructions, opcode distributions, packer signatures, or section layout [70–72,77]. Our task is structurally different on three counts. First, the input is not executable code but a numeric tensor of float32 model weights, whose pixel-level texture is a property of the trained model itself, not of any compiled program; the byte-to-grayscale recipe used by prior work has no semantic anchor in this medium. Second, the signal we are looking for is not a property of the cover but a small, globally distributed statistical perturbation in the mantissa LSBs induced by X-LSB-Attack-Fill (Section 2.1); this is steganalysis, not malware classification, and the relevant prior art is statistical steganalysis of model weights [23,38] rather than malware-image transfer learning. Third, our Grayscale-Fourpart construction is therefore not a transposition of bytecode visualization to a new domain: the 4 × 8-bit segmentation is chosen specifically to expose the float32 mantissa region (see Appendix A.2), where the steganographic modification lives, in alignment with the structural decomposition of the cover medium itself (see Section 3.3.1) - analogous in spirit to Xiao et al.'s [77] use of section-aware overlays for PE files, but on a fundamentally different cover.

The closest methodological analogue in the table is Hsiao et al. [36], which also pairs grayscale images with a Siamese few-shot classifier; the distinction is precisely the one above - input medium (executable bytes vs. float32 weights) and detection signal (instruction-encoding texture vs. mantissa-LSB perturbation). The few-shot setting is, for us, a practical consequence of this medium difference: real attacked-model samples are scarce in a way that real malware binaries are not, and the metric-learning route inherited from [36] is what makes detection feasible in this regime.

7. Practical deployment analysis

A critical requirement for any steganalysis system is practical deployability: the ability to scan models efficiently at scale in real-world scenarios such as model repositories (e.g., Hugging Face, TensorFlow Hub) or organizational model intake pipelines. In this section, we evaluate the computational efficiency of our proposed methods and demonstrate their suitability for production deployment.

7.1. Experimental setup

We conducted performance benchmarks on a representative server configuration:

- **CPU:** Intel(R) Core(TM) i9-11900KF @ 3.50GHz, 16 cores
- **RAM:** 32 GB
- **GPU:** NVIDIA GeForce RTX 3080
- **Framework:** TensorFlow 2.14

7.2. Feature extraction performance

The first stage of our detection pipeline is converting model weights to Grayscale-Fourpart image representations (Section 3.3). Table 4 presents the feature-extraction runtime and Table 5 the memory requirement, across model sizes from 10^2 to 10^8 parameters, comparing our Grayscale-Fourpart features against the reproduced statistical features of B2 (Yin et al. [38]). Since Yin et al. did not release their code, we re-implemented their method following the recipe given in B2 of Section 4.1.1.

As shown in Table 4, our Grayscale-Fourpart representation exhibits excellent scalability. For models with 10^8 parameters (100M, representative of large CNNs like ResNet152, DenseNet201, etc.), feature extraction requires only **0.49 s** on average. In contrast, the baseline statistical feature extraction requires 173.23 s - a ∼**352× slowdown**. This dramatic difference becomes prohibitive for practical deployment: scanning 1000 models would take approximately 8 min with our method versus 48 h with the baseline. The gap widens further at LLM scale, where parameter counts of 10^9 or more are routine.

Memory consumption (Table 5) shows similar advantages. Our method requires 1.77 GB for processing 10^8 parameter models, while the baseline requires 7.25 GB - a 4× difference. This enables deployment on commodity hardware and allows parallel processing of multiple models.

7.3. Inference performance

After feature extraction, the image representations are fed into our trained FSL models (OSL CNN or SRNet) for classification. Table 6 presents single-model inference latency for different model sizes and FSL architectures.

For a single sample, inference takes approximately **0.025 s** with either architecture and evaluation method. OSL CNN shows slightly better performance than SRNet for smaller models due to its simpler architecture (4 convolutional layers vs. SRNet's deep residual structure), but the difference diminishes for large models where image resizing (to 100 × 100 for OSL CNN, 256 × 256 for SRNet) becomes the dominant factor.

7.4. End-to-end performance

Combining feature extraction and inference, the total time to scan a single 100M parameter model is:

$$T_{\text{total}} = T_{\text{feature}} + T_{\text{inference}} \approx 0.49 + 0.025 = 0.515 \text{ s}$$

These numbers demonstrate that our method can handle the scale of major model repositories. For reference, at approximately 0.5 s per model, we can process about 7000 models per hour using single-threaded sequential processing.

Table 4
Runtime in seconds vs. amount of parameters of our suggested image representation and the reproduced baseline statistical features [38]. We report average results over 5 repeated runs and standard deviation.

# Parameters Features	10^2	10^3	10^4	10^5	10^6	10^7	10^8
Ours - GF	0.000125 ± 0.000040	0.000142 ± 0.000094	0.000181 ± 0.000071	0.000639 ± 0.000148	0.004992 ± 0.000714	0.055538 ± 0.007434	0.491864 ± 0.013689
Baseline [38] (Reproduction)	0.012196 ± 0.000628	**0.069653** ± 0.057468	**0.390726** ± 0.456187	**2.699178** ± 3.926365	**4.997636** ± 4.202460	**21.759532** ± 4.545021	**173.232162** ± 0.836762

Table 5
Maximum memory (mebibytes (MiB)) used vs. amount of parameters of our suggested image representation and the reproduced baseline statistical features [38]. We report average results over 5 repeated runs and standard deviation.

# Parameters Features	10^2	10^3	10^4	10^5	10^6	10^7	10^8
Ours - GF	782.18 ± 7.28	782.45 ± 6.31	779.05 ± 7.18	777.99 ± 6.29	787.28 ± 1.35	914.33 ± 2.77	1773.32 ± 35.43
Baseline [38] (Reproduction)	782.22 ± 7.21	782.45 ± 6.31	779.51 ± 6.65	783.72 ± 4.60	**839.82** ± 3.53	**1470.33** ± 18.61	**7247.23** ± 17.59

Table 6
Runtime (seconds) of our suggested methods. We report average results over 10 repeated runs and standard deviation.

	OSL CNN		SRNet	
	Centroid	NN	Centroid	NN
Batch Size				
1	0.0259 ± 0.0459	0.0144 ± 0.0013	0.0226 ± 0.0821	0.0129 ± 0.0028
16	0.044 ± 0.0333	0.0355 ± 0.0013	0.0563 ± 0.1083	0.0222 ± 0.0002
64	0.0451 ± 0.0015	0.0485 ± 0.0014	0.0812 ± 0.0013	0.0816 ± 0.0007
256	0.0884 ± 0.0015	0.099 ± 0.0018	0.3156 ± 0.0052	0.3176 ± 0.0029
1024	0.2609 ± 0.0021	0.2813 ± 0.0059	1.2378 ± 0.0082	1.2544 ± 0.0098

7.5. Comparison with baseline deployability

While the baseline method by Yin et al. [38] achieves competitive detection accuracy (as shown in Section 4.8.4), its computational cost renders it impractical for real-world deployment. The 173-second feature extraction time per 100M model would limit throughput to approximately 20 models/hour on a single core-insufficient for repository-scale deployment.

More critically, the $O(n \log n)$ complexity of statistical feature extraction means that processing time grows super-linearly with model size. As models continue to grow (LLMs with billions of parameters are increasingly common), the baseline approach becomes increasingly infeasible, while our $O(n)$ image conversion scales proportionally.

7.6. Conclusion on practical deployability

Our experimental results demonstrate that the proposed steganalysis method meets the critical requirement of practical deployability. With sub-second end-to-end latency for typical CNN models, low memory footprint, linear scaling properties, and the ability to achieve repository-scale throughput with modest parallelization, our method can be realistically deployed in production environments for real-time model scanning. This practical efficiency, combined with the detection performance demonstrated in previous sections, positions our approach as a viable solution for protecting AI model supply chains from steganographic attacks.

8. Conclusion

In this work, we propose a few-shot learning approach to detecting AI-model steganography attacks. These attacks leverage the AI model-sharing community for malicious purposes by hiding malware in the shared models, and protecting end-users from unknowingly downloading malicious content is a fierce challenge.

The core technical contributions of the paper are: (i) a parameter-position-stable Grayscale-Fourpart (GF) image representation of float32 weights (Section 3.3), motivated by an explicit design rationale that contrasts the image route with the raw-byte / text alternatives (Section 3.3.1); (ii) a few-shot metric-learning steganalysis pipeline built on OSL CNN and SRNet feature extractors with triplet loss and centroid / 1-NN evaluation, trainable from as few as 6 model files (Section 3.4); (iii) a Weighted Metric that aligns evaluation with the practical detection objective by prioritizing low-embedding-rate attacks (Section 4.6).

Empirically, our experimentation achieves very high detection rates using just 6 model files for training, compared to 40,000 samples in our prior work [23] and 100 in Yin et al. [38], and the trained detectors consistently flag delicate attacks down to 25% embedding rate and even 6% in some cases - while [23] was only shown to be effective at 100%-50% embedding rates. We benchmark against a B1-B7 baseline matrix (Section 4.1) spanning two prior academic methods, the canonical raw-byte 1D-CNN paradigm (MalConv-lite), and four threshold-based statistical detectors, and identify the conjoint conditions (architectural homogeneity, payload byte-level structure, abundant training data) under which the simpler baselines succeed and under which they collapse - none of which can be assumed in realistic deployment, where our few-shot detectors retain non-trivial accuracy. The same trained detectors transfer to novel out-of-distribution spread-spectrum attacks (MaleficNet [14]) despite training only on X-LSB-Fill perturbations; we trace the OSL-CNN-vs-SRNet ID/OOD ranking inversion to a concrete mechanism contrast between the two architectures' source-paper design choices (Section 4.8, Experiment 2.5).

Finally, we conduct an end-to-end deployment-feasibility study (Section 7) showing that GF feature extraction and FSL inference scale linearly to 10^8 parameters at ≈ 0.49 s and ~ 1.77 GiB peak memory, with a $\sim 352\times$ runtime advantage over the strongest prior baseline at the same scale - making this, to our knowledge, the first AI-model steganalysis pipeline practical for repository-scale deployment. The full framework, including data creation, X-LSB-Fill and MaleficNet attack replication, FSL training and evaluation, and baseline reproductions B1-B7, is released open-source [40] to support reproducibility and to seed further research in this new field.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used ChatGPT in order to edit the texts (rephrasing, etc). After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

CRediT authorship contribution statement

Daniel Gilkarov: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Ran Dubin:** Validation, Supervision, Resources, Project administration, Methodology, Conceptualization.

Data availability

The full pipeline used in this work - data creation, X-LSB-Fill and MaleficNet attack replication, image representation, FSL training and evaluation, and the B1-B7 baseline reproductions - is openly released as software framework [40]. Pre-trained model zoos used as benign references are obtained from publicly available sources (Keras applications and the GHRP zoos of [56]); attacked variants are re-generated by the released framework rather than redistributed, since the attacked weights would otherwise duplicate copies of the original model files with malware payloads embedded.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Appendix A. Background

A.1. X-LSB-Attack

We include pseudo-code to illustrate better the 2 X-LSB-Attack variants described in Section 2.1. In the pseudo-code, we use $w[0:r]$ to denote the first r bits of the float number w. $v+u$ denotes the concatenation of 2 binary strings, and v^c denotes concatenating v to itself c times.

Algorithm 2 X - LSB attack.

Input: Integer $1 \leq X \leq s$,
List of float numbers $W := [w_1, w_2, \dots, w_n]$,
binary malware $m := b_1 \cdots b_k$

Output: List of float numbers $\widehat{W}$ with m embedded in its numbers.

if $k > n \cdot X$ **then**
    m cannot fully fit inside W, terminate.
end if
Set $t := \lceil k/X \rceil$
Let $m_i := b_{(i-1)X+1} \cdots b_{iX}\ \forall 1 \leq i < t$
$m_{remainder} = m_t := b_{(t-1)X+1} \cdots b_k$
Set $\widehat{w_i} := w_i[0 : s-X] + m_i\ \forall 1 \leq i < t$
Set $\widehat{w_t} := w_t[0 : s-X] + m_t$
Return $\widehat{W} := [\widehat{w_1}, \dots, \widehat{w_t}, w_{t+1}, \dots, w_n]$

Algorithm 3 X - LSB attack - Fill.

Input: Integer $1 \leq X \leq s$,
List of float numbers $W := [w_1, w_2, \dots, w_n]$,
binary malware $m := b_1 \cdots b_k$

Output: List of float numbers $\widehat{W}$ with m fully embedding its X-LSBs.

if $k > n \cdot X$ **then**
    Set $\hat{m} := m[0 : nX]$
else
    Set $\hat{m} := m^{\lceil nX/k \rceil}[0 : nX]$
end if
Return X-LSB-Attack(X, W, $\hat{m}$)

A.2. Floating point format

Floating-point numbers [43] (also called floats) represent decimal numbers in computer memory. Float numbers are divided into three sections: sign, exponent, and mantissa (also called fraction). The sign bit denotes the sign of the number, the exponent section represents a negative or positive power of 2, and the mantissa section represents the precise value after the decimal point. The data type we use throughout this paper is single-precision float (float32), which takes up 32 bits in memory; we explain its layout next.

A.2.1. Float32

Float32 takes up 32 bits in memory, as the name suggests. The 32 bits are segmented into 1/8/23 portions for the sign/exponent/mantissa sections. For some binary float32 no. $b_{31}b_{30} \cdots b_1 b_0$ where b_{31} is the sign bit, $b_{30}b_{29} \cdots b_{23}$ is the exponent section and $b_{22}b_{21} \cdots b_0$ is the mantissa section, the decimal value of the number is calculated by

$$(-1)^{b_{31}} \cdot 2^{(b_{30}b_{29}\cdots b_{23})_2 - 127_{10}} \cdot (1.b_{22}b_{21} \cdots b_0)_2$$

See Fig. A.1 for an illustration.



Fig. A.1. Illustration of float32 numbers. In this example, the sign bit b_{31} equals $0_2 = 0_{10}$, the exponent section $b_{30}b_{29} \cdots b_{23}$ equals $01111100_2 = 124_{10}$, and the mantissa region $b_{22}b_{21} \cdots b_0$ equals $01 \cdots 0_2$. Hence, the number equals $(-1)^{b_{31}} \cdot 2^{(b_{30}b_{29}\cdots b_{23})_2} \cdot (1.b_{22}b_{21} \cdots b_0)_2 = (-1)^0 \cdot 2^{124-127} \cdot (1.01 \cdots 0)_2 = +2^{-3} \cdot 1.25 = 0.15625$.

Appendix B. Model performance

In this section, we present model evaluation results of benign and attacked models from our created datasets (Section 4.4) to illustrate how the steganography attacks affect the model performance on some validation dataset. The models from the SCZ dataset are each tested on their corresponding dataset, and the models from the famous CNN architectures are tested against the ImageNet2012 [78] validation dataset. See Tables B.1–B.3. We can see that when compared with the benign model results, attacks up to 16 bits (50% embedding rate) cause almost no degradation in model performance. This means attackers would opt to leverage 1–16 bits of the parameters to avoid causing suspicion in end-users. Nevertheless, even 1–16 bits, which are 3.1%-50% of the whole model, allow an attacker an embedding capacity of 3MB-50MB in a 100MB model. In addition, we can see that using 22 and 23 bits in the

Table B.1
Benign and attacked average SCZ model performance on the corresponding validation dataset.

ZOO\X	0 (benign)	8	16	20	21	22	23
CIFAR10	**45**	**45**	**45**	**45**	43	39	30
MNIST	**81**	**81**	**81**	**81**	80	72	64
STL10	**34**	**34**	**34**	**34**	33	30	25
SVHN	**65**	**65**	**65**	**65**	63	56	43

Table B.2
Inference accuracy of famous small CNNs before and after steganography attack. Models were tested on the ImageNet2012 validation dataset. We see that attacks that used 1–16 bits had almost no effect.

ZOO\X	0 (benign)	8	16	20	21	22	23
DenseNet121	**74**	**74**	**74**	71	55	9	0
EfficientNetV2B0	**78**	**78**	**78**	75	49	2	0
MobileNet	**70**	**70**	**70**	34	1	0	0
MobileNetV2	**71**	**71**	70	29	0	0	0
MobileNetV3Small	**68**	**68**	67	5	0	0	0
MobileNetV3Large	**75**	**75**	**75**	35	1	0	0
NasNetMobile	**74**	**74**	**74**	69	30	0	0

Table B.3
Inference accuracy of famous large CNNs before and after steganography attack. Models were tested on the ImageNet2012 validation dataset.

ZOO\X	0 (benign)	8	16	20	21	22	23
DenseNet169	**76**	**76**	**76**	73	60	3	0
DenseNet201	**77**	**77**	**77**	73	61	1	0
EfficientNetV2B2	**80**	**80**	**80**	77	53	0	0
EfficientNetV2B3	**82**	**82**	**82**	79	63	1	0
InceptionResNetV2	**80**	**80**	**80**	78	68	4	0
InceptionV3	**77**	**77**	**77**	75	60	4	0
ResNet50	**74**	**74**	**74**	70	55	11	0
ResNet50V2	**70**	**70**	**70**	66	38	2	0
ResNet101	**76**	**76**	**76**	73	61	8	0
ResNet101V2	**72**	**72**	**72**	66	38	1	0
ResNet152	**76**	**76**	**76**	67	48	1	0
ResNet152V2	**72**	**72**	**72**	68	54	5	0
Xception	**79**	**79**	**79**	74	43	0	0

attack completely eroded the models in all cases. We can directly conjecture that attacks outside the mantissa will be even more noticeable and unviable for attackers since the exponent bits in float32 can cause exponential changes (see Appendix A.2). This can justify focusing only on the mantissa.

References

[1] Ouyang W, Zeng X, Wang X, Qiu S, Luo P, Tian Y, et al. DeepID-net: object detection with deformable part based convolutional neural networks. IEEE Trans Pattern Anal Mach Intell 2017;39(7):1320–34. https://doi.org/10.1109/TPAMI.2016.2587642

[2] Noh H, Hong S, Han B. Learning deconvolution network for semantic segmentation. In: 2015 IEEE International conference on computer vision (ICCV). Los Alamitos, CA, USA: IEEE Computer Society; 2015, p‘p. 1520–8. https://doi.org/10.1109/ICCV.2015.178

[3] Devlin J, Chang M-W, Lee K, Toutanova K. BERT: pre-training of deep bidirectional transformers for language understanding. 2019. arXiv:1810.04805

[4] Punjani A, Abbeel P. Deep learning helicopter dynamics models. In: 2015 IEEE International conference on robotics and automation (ICRA). 2015, pp. 3223–30. https://doi.org/10.1109/ICRA.2015.7139643

[5] Yang Y, Li Y, Fermüller C, Aloimonos Y. Robot learning manipulation action plans by “watching” unconstrained videos from the world wide web. In: Proceedings of the twenty-ninth AAAI conference on artificial intelligence (AAAI’15). 2015, pp. 3686–92.

[6] OpenAI. ChatGPT: a large-scale generative model for conversation, 2022. https://openai.com/research/chatgpt;

[7] HuggingFace. Fickling pickle scanning. 2022. Accessed: 2022-12-19, https://huggingface.co/.

[8] TensorFlow. TensorFlow hub. 2022. Accessed: 2022-12-19, https://www.tensorflow.org/hub.

[9] Jeddi A, Shafiee MJ, Wong A. A simple fine-tuning is all you need: towards robust deep learning via adversarial fine-tuning. 2020. arXiv:2012.13628

[10] Wang Z, Liu C, Cui X. EvilModel: hiding malware inside of neural network models. In: 2021 IEEE Symposium on computers and communications (ISCC). IEEE; 2021, p. 1–7.

[11] Wang Z, Liu C, Cui X, Yin J, Wang X. EvilModel 2.0: bringing neural network models into malware attacks. Comput Secur 2022;120:102807.

[12] Liu T, Liu Z, Liu Q, Wen W, Xu W, Li M. StegoNet: turn deep neural network into a stegomalware. In: Annual computer security applications conference. ACSAC ’20; New York, NY, USA: Association for Computing Machinery. 2020, pp. 928–38. https://doi.org/10.1145/3427228.3427268

[13] Verma A. Insecure deserialization detection in python. Master’s project; San José State University; 2023. https://doi.org/10.31979/etd.3yzt-6hxp

[14] Hitaj D, Pagnotta G, Hitaj B, Mancini LV, Perez-Cruz F. MaleficNet: hiding malware into deep neural networks using spread-spectrum channel coding. In: Computer security – ESORICS 2022: 27th European symposium on research in computer security, Copenhagen, Denmark, September 26–30, 2022, Proceedings, Part III. Berlin, Heidelberg: Springer-Verlag. 2022, pp. 425–44. https://doi.org/10.1007/978-3-031-17143-7_21

[15] CyberTalk. Hackers hide malware in windows logo, 2022. https://www.cybertalk.org/2022/09/30/hackers-hide-malware-in-windows-logo/;

[16] Cimpanu C. WAV audio files are now being used to hide malicious code. https://www.zdnet.com/article/wav-audio-files-are-now-being-used-to-hide-malicious-code/; 2019.

[17] Wang Z, Byrnes O, Wang H, Sun R, Ma C, Chen H, et al. Data hiding with deep learning: a survey unifying digital watermarking and steganography. arXiv:2107.09287 2021.

[18] Subramanian N, Elharrouss O, Al-Maadeed S, Bouridane A. Image steganography: a review of the recent advances. IEEE Access 2021;9:23409–23.

[19] AlSabhany AA, Ali AH, Ridzuan F, Azni AH, Mokhtar MR. Digital audio steganography: systematic review, classification, and analysis of the current state of the art. Comput Sci Rev 2020;38:100316.

[20] Liu Y, Liu S, Wang Y, Zhao H, Liu S. Video steganography: a review. Neurocomputing 2019;335:238–50.

[21] Majeed MA, Sulaiman R, Shukur Z, Hasan MK. A review on text steganography techniques. Mathematics 2021;9(21):2829.

[22] Bansal K, Agrawal A, Bansal N. A survey on steganography using least significant bit (LSB) embedding approach. In: 2020 4th international conference on trends in electronics and informatics (ICOEI)(48184). IEEE; 2020, pp. 64–9.

[23] Gilkarov D, Dubin R. Steganalysis of AI models LSB attacks. IEEE Trans Inf Forensics Secur 2024;19:4767–79. https://doi.org/10.1109/TIFS.2024.3383770

[24] Dubin R. Disarming attacks inside neural network models. IEEE Access 2023;11:124295–303. https://doi.org/10.1109/ACCESS.2023.3330141

[25] Amit G, Levy M, Mirsky Y. Transpose attack: stealing datasets with bidirectional training. 2023. arXiv:2311.07389

[26] Neyshabur B, Bhojanapalli S, McAllester D, Srebro N. Exploring generalization in deep learning. 2017. arXiv:1706.08947

[27] Gilkarov D, Dubin R. AI model steganalysis. https://github.com/ArielCyber/AI_Model_Steganalysis; 2023.

[28] Fei-Fei L, Fergus R, Perona P. One-shot learning of object categories. IEEE Trans Pattern Anal Mach Intell 2006;28(4):594–611. https://doi.org/10.1109/TPAMI.2006.79

[29] Shaban A, Bansal S, Liu Z, Essa I, Boots B. One-shot learning for semantic segmentation. 2017. arXiv:1709.03410

[30] Bateni P, Goyal R, Masrani V, Wood F, Sigal L. Impiroved few-shot visual classificaton. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR). 2020.

[31] Zhou J, Zheng Y, Tang J, Li J, Yang Z. FlipDA: effective and robust data augmentation for few-shot learning. 2022. arXiv:2108.06332

[32] Nakamura A, Harada T. Revisiting fine-tuning for few-shot learning. 2019. arXiv:1910.00216

[33] Liu C, Fu Y, Xu C, Yang S, Li J, Wang C, et al. Learning a few-shot embedding model with contrastive learning. Proc AAAI Conf Artif Intell 2021;35(10):8635–43. https://doi.org/10.1609/aaai.v35i10.17047

[34] Kaya M, Bilge HŞ. Deep metric learning: a survey. Symmetry 2019;11(9):1066.

[35] Koch G, Zemel R, Salakhutdinov R, et al. Siamese neural networks for one-shot image recognition. In: ICML deep learning workshop. Lille; 2015,.

[36] Hsiao S-C, Kao D-Y, Liu Z-Y, Tso R. Malware image classification using one-shot learning with siamese networks. Procedia Comput Sci 2019;159:1863–71. Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 23rd International Conference KES2019. https://doi.org/10.1016/j.procs.2019.09.358

[37] Boroumand M, Chen M, Fridrich J. Deep residual network for steganalysis of digital images. IEEE Trans Inf Forensics Secur 2019;14(5):1181–93. https://doi.org/10.1109/TIFS.2018.2871749

[38] Yin Y, Zhang W, Yu N, Chen K. Steganalysis of neural networks based on parameter statistical bias. JUSTC 2022;52(1):1.

[39] Raff E, Barker J, Sylvester J, Brandon R, Catanzaro B, Nicholas C. Malware detection by eating a whole EXE. In: The workshops of the thirty-second AAAI conference on artificial intelligence. 2018, pp. 268–76.

[40] Gilkarov D, Dubin R. Model X-ray code repository. https://github.com/danigil/ModelXRay; 2026. Accessed: 2026-04-28.

[41] Johnson NF, Jajodia S. Exploring steganography: seeing the unseen. Computer 1998;31(2):26–34. https://doi.org/10.1109/MC.1998.4655281

[42] Bhattacharyya D, Dutta P, Balitanas MO, Kim T-h, Das P. Hiding data in audio signal. In: Chang C-C, Vasilakos T, Das P, Kim T-h, Kang B-H, Khurram Khan M, editors. Advanced communication and networking. Berlin, Heidelberg: Springer. 2010, pp. 23–9.

[43] Wikipedia contributors. Single-precision floating-point format — wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Single-precision_floating-point_format&oldid=1220802754; 2024. [Online; accessed 20-May-2024].

[44] MalwareBazaar. Malware sharing platform. 2024. Accessed: 2024-05-14; https://bazaar.abuse.ch/.

[45] Pal KK, Sudeep KS. Preprocessing for image classification by convolutional neural networks. In: 2016 IEEE International conference on recent trends in electronics, information & communication technology (RTEICT). 2016, pp. 1778–81. https://doi.org/10.1109/RTEICT.2016.7808140

[46] Schroff F, Kalenichenko D, Philbin J. FaceNet: a unified embedding for face recognition and clustering. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2015, pp. 815–23.

[47] Kingma DP, Ba J. Adam: a method for stochastic optimization. 2017. arXiv:1412.6980

[48] Wang Y, Chao W-L, Weinberger KQ, van der Maaten L. SimpleShot: revisiting nearest-neighbor classification for few-shot learning. 2019. arXiv:1911.04623

[49] Chen T, Guestrin C. XGBoost: a scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining (KDD). 2016, pp. 785–94. https://doi.org/10.1145/2939672.2939785

[50] Rukhin A, Soto J, Nechvatal J, Smid M, Barker E, Leigh S, et al. A statistical test suite for random and pseudorandom number generators for cryptographic applications. NIST Special Publication 800-22 Rev. 1a; National Institute of Standards and Technology (NIST); Gaithersburg, MD, USA; 2010. https://doi.org/10.6028/NIST.SP.800-22r1a

[51] Knuth DE. The art of computer programming, volume 2: seminumerical algorithms. Reading, MA: Addison-Wesley; 3rd ed.; 1997.

[52] Lyda R, Hamrock J. Using entropy analysis to find encrypted and packed malware. IEEE Secur Priv 2007;5(2):40–5. https://doi.org/10.1109/MSP.2007.48

[53] Cachin C. An information-theoretic model for steganography. Inf Comput 2004;192(1):41–56. https://doi.org/10.1016/j.ic.2004.02.003

[54] Farid H. Detecting hidden messages using higher-order statistical models. In: Proceedings of the international conference on image processing (ICIP); vol. 2. 2002, pp. 905–8. https://doi.org/10.1109/ICIP.2002.1040098

[55] Fridrich J. Feature-based steganalysis for JPEG images and its implications for future design of steganographic schemes. In: Information hiding: 6th international workshop (IH 2004); vol. 3200 of *LNCS*. Springer; 2004, pp. 67–81. https://doi.org/10.1007/978-3-540-30114-1_6

[56] Schürholt K, Taskiran D, Knyazev B, Giró-i Nieto X, Borth D. Model zoos: a dataset of diverse populations of neural network models. 2022. arXiv:2209.14764

[57] Keras. Keras applications. 2024. Accessed: 2024-04-13; https://keras.io/api/applications/.

[58] Huang G, Liu Z, van der Maaten L, Weinberger KQ. Densely connected convolutional networks. 2018. arXiv:1608.06993

[59] Tan M, Le QV. EfficientNetv2: smaller models and faster training. 2021. arXiv:2104.00298

[60] Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, et al. MobileNets: efficient convolutional neural networks for mobile vision applications. 2017. arXiv:1704.04861

[61] Sandler M, Howard A, Zhu M, Zhmoginov A, Chen L-C. MobileNetv2: inverted residuals and linear bottlenecks. 2019. arXiv:1801.04381

[62] Howard A, Sandler M, Chu G, Chen L-C, Chen B, Tan M, et al. Searching for MobileNetv3. 2019. arXiv:1905.02244

[63] Zoph B, Vasudevan V, Shlens J, Le QV. Learning transferable architectures for scalable image recognition. 2018. arXiv:1707.07012
[64] Szegedy C, Vanhoucke V, Ioffe S, Shlens J, Wojna Z. Rethinking the inception architecture for computer vision. 2015. arXiv:1512.00567
[65] Szegedy C, Ioffe S, Vanhoucke V, Alemi A. Inception-v4, inception-resnet and the impact of residual connections on learning. 2016. arXiv:1602.07261
[66] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. 2015. arXiv:1512.03385
[67] He K, Zhang X, Ren S, Sun J. Identity mappings in deep residual networks. 2016. arXiv:1603.05027
[68] Chollet F. Xception: deep learning with depthwise separable convolutions. 2017. arXiv:1610.02357
[69] Waskom ML. Seaborn: statistical data visualization. J Open Source Softw 2021;6(60):3021. https://doi.org/10.21105/joss.03021
[70] Nataraj L, Karthikeyan S, Jacob G, Manjunath BS. Malware images: visualization and automatic classification. In: Proceedings of the 8th international symposium on visualization for cyber security. VizSec '11; New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2016904.2016908
[71] Gibert D, Mateu C, Planes J. Using convolutional neural networks for classification of malware represented as images. J Comput Virol Hacking Tech 2019;15(1):15–28.
[72] Kalash M, Rochan M, Mohammed N, Bruce N DB, Wang Y, Iqbal F. Malware classification with deep convolutional neural networks. In: 2018 9th IFIP international conference on new technologies, mobility and security (NTMS). 2018, pp. 1–5. https://doi.org/10.1109/NTMS.2018.8328749
[73] Vasan D, Alazab M, Wassan S, Safaei B, Zheng Q. Image-based malware classification using ensemble of CNN architectures (IMCEC). Comput Secur 2020;92:101748. https://doi.org/10.1016/j.cose.2020.101748
[74] Zhao Z, Zhao D, Yang S, Xu L. Image-based malware classification method with the alexnet convolutional neural network model. Secur Commun Netw 2023;2023(1):6390023. https://doi.org/10.1155/2023/6390023
[75] Chaganti R, Ravi V, Pham TD. Image-based malware representation approach with efficientnet convolutional neural networks for effective malware classification. J Inf Secur Appl 2022;69:103306. https://doi.org/10.1016/j.jisa.2022.103306
[76] Demmese FA, Neupane A, Khorsandroo S, Wang M, Roy K, Fu Y. Machine learning based fileless malware traffic classification using image visualization. Cybersecurity 2023;6(1):32.
[77] Xiao M, Guo C, Shen G, Cui Y, Jiang C. Image-based malware classification using section distribution information. Comput Secur 2021;110:102420. https://doi.org/10.1016/j.cose.2021.102420
[78] Russakovsky O, Deng J, Su H, Krause J, Satheesh S, Ma S, et al. ImageNet large scale visual recognition challenge. Int J Comput Vis 2015;115(3):211–52. https://doi.org/10.1007/s11263-015-0816-y



Daniel Gilkarov received his BSc and MSc computer science degrees from Ariel University, Israel. He is currently pursuing a PhD degree in computer and software engineering at Ariel University. His research focuses on AI security, defending the machine learning model supply chain, and other cybersecurity applications.



Ran Dubin received his BSc, MSc, and PhD degrees from Ben-Gurion University, Beer Sheva, Israel, all in communication systems engineering. He is currently a faculty member at the Computer Science Department, Ariel University, Israel. His research interests revolve around zero-trust cyber protection, malware disarms and reconstruction, encrypted network traffic detection, Deep Packet Inspection (DPI), bypassing AI, Natural Language Processing, and AI trust.